

1. INTRODUCTION

1. Introduction

In the modern digital world, where technology has become the backbone of every sector, securing data has become one of the greatest challenges faced by individuals, organizations, and governments. With the rapid growth of the Internet and social media, sensitive information such as personal identities, financial records, defense data, and medical reports is constantly being shared and transmitted across digital networks. While this connectivity has created convenience and global reach, it has also made digital data highly vulnerable to unauthorized access, modification, theft, and misuse.

Cybercrime is no longer limited to hacking or phishing; it now involves sophisticated attacks that can intercept or manipulate digital data during transmission. Traditional encryption methods like AES, RSA, and DES help protect the contents of data, but they do not conceal the existence of the message. If encrypted data is detected, attackers may attempt brute-force or cryptanalysis attacks to retrieve it. This is where steganography the science of hiding information within other seemingly innocent digital objects plays a vital role.

Steganography conceals secret messages within cover media such as images, audio, or video files, such that an observer cannot detect any alteration. When combined with cryptography, it provides a two-layer defense mechanism: first encrypting the message to make it unreadable, and then embedding it into a cover file to make its existence undetectable. This dual mechanism significantly enhances data security, ensuring that even if the communication is intercepted, the attacker cannot recognize or extract the hidden content. The proposed system, Data Steganography for Image and Audio Using Deep Genetic Algorithm and LSB, focuses on developing a highly secure and efficient data hiding mechanism.

It integrates cryptographic encryption with intelligent optimization techniques to improve both security and performance. The system employs the Least Significant Bit (LSB) technique for embedding data and enhances it using a Genetic Algorithm (GA) to select optimal pixels for data insertion. Before embedding, the secret data is encrypted using the Advanced Encryption Standard (AES) algorithm, ensuring complete confidentiality.

2. Overview

The project —Data Steganography for Image and Audio Using Deep Genetic Algorithm and LSB focuses on designing a hybrid computational model for secure data hiding and transmission by integrating cryptographic encryption, steganographic embedding, and evolutionary optimization techniques. With the exponential growth of digital communication, the need to protect sensitive data such as financial records, defense information, and personal credentials has become a major concern. Traditional cryptographic techniques like AES or RSA ensure confidentiality but fail to disguise the very existence of the message, which can still attract malicious attention. To overcome this limitation, the proposed system combines Least Significant Bit (LSB) steganography a method for embedding data into the least noticeable parts of digital images with the optimization capabilities of a Genetic Algorithm (GA) to improve data imperceptibility and security.

Before embedding, the confidential data is first encrypted using the Advanced Encryption Standard (AES), which performs multiple rounds of substitution, permutation, and key expansion to convert the plaintext into ciphertext. This ciphertext is then hidden inside a cover image using the LSB substitution method, where selected pixel bits are replaced by encrypted data bits in a way that preserves image quality. The GA component enhances this process by selecting the most optimal pixel positions for data embedding, reducing statistical detectability and visual distortion.

During extraction, the hidden bits are retrieved from the stego-image, decrypted using the AES key, and converted back into the original message. This multi-level framework ensures that even if the stego-image is intercepted, the concealed data remains secure and unreadable without both the extraction and decryption keys. The system provides a comprehensive security model characterized by confidentiality, robustness, and high imperceptibility, making it highly applicable in domains such as military communication, healthcare data protection, and digital watermarking. Furthermore, the combination of artificial intelligence-driven optimization and conventional cryptography demonstrates how modern computational methods can enhance traditional security frameworks, ensuring that data remains private, reliable, and tamper-proof in a world increasingly reliant on digital information exchange.

3. Problem Statement

To develop and design a Secure Data Transmission System Using Data Steganography System.

4. Objectives

- To study and analyze existing methods of image and audio steganography.
- To design a secure model combining AES encryption, LSB embedding, and Genetic Algorithm optimization.
- To implement steganography for hiding text inside both images and audio files.
- To ensure data integrity and imperceptibility of hidden content.
- To evaluate performance using metrics such as PSNR, MSE, and data recovery accuracy.
- To develop a user-friendly Java-based interface for encoding and decoding hidden

5. Importance of Android Applications

Web-based applications play a crucial role in modern digital systems, providing benefits across personal, business, and societal levels.

1. Secure Data Transmission

The system provides a highly secure way of transmitting sensitive information by combining AES encryption and LSB steganography. Even if data is intercepted, it remains hidden and unreadable, ensuring confidentiality.

2. Remote Accessibility

Being a web-based application, users can securely upload, encode, and decode data from anywhere using a browser. This eliminates the need for installing software and allows access across multiple devices.

3. Dual-Layer Security Mechanism

The integration of encryption + steganography provides two levels of protection:

- Encryption hides the content
- Steganography hides the existence of data

This makes the system more robust against cyber attacks.

4. Efficient Handling of Multimedia Data

The system supports both image and audio files for data hiding. This flexibility allows users to choose different media formats while maintaining high data security and minimal distortion.

5. Intelligent Optimization using Genetic Algorithm

The use of a Genetic Algorithm (GA) helps in selecting optimal positions for data embedding. This improves:

- Security
- Image/audio quality
- Resistance to detection

6. Global and Regional Context of Web-Based Secure Data Transmission Systems

Global Context

1. Growing Need for Data Security

With the rapid increase in digital communication, protecting sensitive information has become a global priority. Web-based secure systems are widely used to prevent data breaches, cyberattacks, and unauthorized access during online data transmission.

2. Expansion of Web Technologies

Web applications are globally adopted due to their accessibility and scalability. Technologies like cloud computing, APIs, and secure web protocols (HTTPS, SSL) enable secure data exchange across platforms, making web-based security systems essential.

3. Cross-Industry Applications

Web-based secure data transmission systems are used in multiple sectors such as:

- Healthcare (secure patient data sharing)
- Banking & Finance (secure transactions and online banking)
- Defense & Military (confidential communication)
- E-commerce (secure payment and user data protection)

4. Role in Cybersecurity Ecosystem

These systems contribute significantly to global cybersecurity by combining techniques like encryption, steganography, and secure protocols, ensuring confidentiality, integrity, and data protection during transmission.

5. Rise of Cloud-Based Security Systems

Modern web applications are integrated with cloud platforms (AWS, Azure, Google Cloud), enabling scalable and secure storage and transmission of data across global networks.

Regional Context

1. Rapid Digital Growth

Countries like India are experiencing rapid digital transformation with increasing internet users. Secure web-based systems are essential for protecting data in online services such as banking, education, and government platforms.

2. Digital India and Online Services

Government initiatives like Digital India promote online services such as:

- DigiLocker
- Online banking
- E-governance portals

These require secure data transmission systems to protect user information.

3. Increasing Cybersecurity Threats

With the rise of online platforms, cyber threats like hacking, phishing, and data theft are increasing. Web-based security systems using encryption and steganography help reduce these risks.

4. Support for Startups and Innovation

India's growing tech ecosystem encourages development of secure web applications. Startups and developers are building solutions for secure communication, data protection, and privacy.

Continent-wise Distribution and Context

1. Asia

- **Adoption Level: Very High**

- **Highlights:**

- Rapid growth of internet users (India, China, Southeast Asia)
- High demand for secure communication systems
- Used in banking, e-commerce, and government services
- Increasing focus on cybersecurity solutions

2. Africa

- **Adoption Level: Growing**

- **Highlights:**

- Web-based systems used for mobile banking and data protection
- Increasing internet penetration
- Need for low-cost, secure communication solutions
- Focus on protecting financial and user data

3. Europe

- **Adoption Level: High**

- **Highlights:**

- Strong data privacy laws (GDPR)
- High demand for secure web applications
- Used in healthcare, finance, and enterprise systems
- Emphasis on data protection and encryption

4. North America

- **Adoption Level: Very High**

- **Highlights:**

- Advanced cybersecurity infrastructure
- High use of secure web systems in enterprises
- Focus on data privacy, cloud security, and encryption
- Strong demand for secure communication platforms

5. South America

- **Adoption Level: Moderate to High**

- **Highlights:**

- Growing digital economy
- Increasing use of web applications for banking and e-commerce
- Rising need for secure data transmission systems

2. LITERATURE SURVEY

1) Title: Secure Data Transfer Over Internet Using Image Steganography (2024)

Authors: Rikeeta Mahajan, Bholenath More, Lina Gunjal, Bhavesh Waghulde, Rohit Narkhede

The paper by Mahajan et al. (2024) focuses on applying image steganography for secure data transmission over public networks, ensuring confidential communication without altering the original data at the receiver's end. Due to the rapid increase in digital communication and frequent data breaches on platforms such as social media and cloud-based applications, the study highlights the importance of concealed data transfer techniques that can protect sensitive information from unauthorized access. The authors emphasize that steganography provides data invisibility, confidentiality, and protection against misuse by attackers.

The study discusses limitations of modern communication systems where personal data is vulnerable despite encryption, as encrypted data still attracts attacker attention. To minimize this risk, the authors apply image-based steganography using the Least Significant Bit (LSB) algorithm, which modifies only the least significant bits of pixels, making embedded data imperceptible to human observation. The LSB approach ensures that only minimal changes occur in the image, preserving its original appearance.

The system workflow includes selecting a cover image, embedding a secret message using LSB encoding, protecting the image with a secret key, and transmitting the resulting stego-image. At the receiving end, decoding requires the correct secret key to extract hidden information. This ensures authenticated access and prevents unauthorized data retrieval. The implementation is carried out using Java in the Apache NetBeans IDE, making the system easy to deploy and accessible for users without expert knowledge in cybersecurity.

The authors list major advantages of the system, including high security, payload capacity, and resilience, making it suitable for confidential communication in government and law enforcement. It also protects against identity theft, unauthorized reading, and data manipulation. However, the approach also has certain limitations—larger stego-image size may raise suspicion, and the hidden

message may become difficult to recover if the image undergoes transformations such as rotation, scaling, or noise distortion.

The study concludes that LSB-based image steganography is a simple yet effective approach for secure communication, particularly in environments where encryption alone is insufficient. Although the method proves reliable for message hiding, future improvements should focus on strengthening robustness against image manipulation attacks. This work contributes a practical use case for secure image-based communication and serves as a foundation for further research in optimized, attack-resistant steganographic systems.

2) Title: Deep Learning and Genetic Algorithm Techniques for Secure Multimedia Steganography (2023)

Authors: Zhang L., Kumar S., Patel R., and Nguyen T.

The paper by Zhang L. et al. (2023) presents a comprehensive overview of advanced multimedia steganography methods that combine deep learning frameworks with evolutionary optimization algorithms to enhance secure data hiding. With the growing prevalence of digital media sharing over public networks, the study emphasizes the need for steganographic systems that preserve the confidentiality of sensitive data while maintaining the integrity of the original media. The authors provide a detailed review of how artificial intelligence, particularly convolutional neural networks (CNNs), encoder-decoder architectures, and genetic algorithms (GAs), is reshaping secure multimedia transmission.

The study begins by discussing the limitations of traditional steganography methods such as basic Least Significant Bit (LSB) substitution, which are vulnerable to statistical analysis and compression attacks. Traditional techniques struggle to adaptively hide data in complex images and audio streams, often resulting in detectable distortions. To overcome these challenges, the authors introduce the use of genetic algorithms for frame selection, adaptive LSB labeling, and pixel optimization. The GA identifies regions of high entropy in cover media, ensuring hidden payloads are embedded with minimal perceptible change.

Additionally, the paper examines the application of deep learning models in steganography. CNNs are primarily used for identifying optimal embedding regions in images by extracting spatial

and sequential features that can carry hidden data without noticeable alterations. Encoder- decoder networks are employed to reconstruct the hidden information accurately during decoding, even in cases of compression or noise, enhancing the robustness of the approach compared to conventional techniques.

A significant contribution of the study is its focus on hybrid approaches that integrate deep learning with genetic algorithms. This combination allows the system to select optimal frames in audio and video, perform adaptive LSB labeling, and incorporate AES encryption to ensure secure communication. The authors also discuss multi-stage embedding and wavelet-based transformations that improve resistance to attacks and maintain high visual and auditory fidelity.

The study concludes that the fusion of evolutionary algorithms and deep learning frameworks represents a significant advancement in multimedia steganography. Such hybrid systems achievement higher payload capacity, greater imperceptibility, and stronger security against steganalysis attacks than traditional methods. The authors suggest that future research will emphasize AI- driven adaptive steganography capable of embedding large volumes of sensitive data across diverse multimedia formats while preserving recoverability under hostile conditions.

Overall, Zhang et al. (2023) provide a strong foundation for understanding current and emerging trends in AI-assisted steganography. Their review informs the current study, which integrates Genetic Algorithms, AES encryption, and LSB-based techniques for secure audio and image steganography, aligning with modern approaches to high-security digital communication and covert data transmission.

3) Title: DeepGA-Stego: Hybrid Deep Learning and Genetic Algorithm Approach for Secure Audio-Image Steganography (2022)

Authors: Kumar S., Li Y., Zhang H., Patel R., and Nguyen T.

The study —DeepGA-Stego: Hybrid Deep Learning and Genetic Algorithm Approach for Secure Audio-Image Steganography by Kumar S., Li Y., Zhang H., Patel R., and Nguyen T. (2022) introduces an innovative framework that combines deep learning and evolutionary optimization to enhance multimedia steganography security. This research represents a significant advancement in

digital data protection, showing how multi-layer neural networks integrated with genetic algorithms can outperform traditional steganographic methods in terms of payload imperceptibility, robustness, and resistance to attacks.

The DeepGA-Stego framework was developed to address a major challenge in secure multimedia communication—the vulnerability of hidden data to detection and distortion in complex audio and image media. Conventional techniques such as standard Least Significant Bit (LSB) substitution or fixed-pattern embedding often produce detectable artifacts, particularly under compression or noise conditions. DeepGA-Stego overcomes these limitations by employing a hybrid architecture that couples convolutional neural networks (CNNs) for feature extraction with genetic algorithms (GAs) for optimized frame and bit selection.

The model architecture consists of two primary modules:

1. **Multimedia feature extractor:** A CNN-based module processes input images and audio to identify high-entropy regions that are suitable for data embedding, extracting spatial, spectral, and temporal features without introducing perceptible distortions.
2. **Genetic optimization module:** This module applies GA-based selection, crossover, and mutation operators to identify the best embedding positions in frames or audio segments. The GA ensures that payloads are distributed optimally across the media, enhancing security and minimizing detectable changes.

These two modules work in tandem, producing an embedding map that guides secure insertion of encrypted secret messages. AES encryption is applied prior to embedding, further strengthening the confidentiality of the hidden data. The framework was evaluated using diverse datasets of images and audio files in different formats (JPEG, PNG, WAV, MP3), ensuring generalization across real-world media. Preprocessing steps such as normalization, bit-level alignment, and noise filtering were implemented to improve embedding accuracy and maintain media fidelity.

Performance analysis revealed that DeepGA-Stego significantly outperformed conventional steganography techniques, including standard LSB and histogram-based methods, achieving higher peak signal-to-noise ratio (PSNR) and lower mean square error (MSE). The hybrid

system also exhibited strong robustness against steganalysis attacks, including statistical and noise-based detection. In addition, payload recovery rates remained high even after lossy compression or transmission through noisy channels, demonstrating the practical applicability of the model.

A key contribution of this paper is its focus on adaptive and interpretable embedding strategies. By analyzing CNN feature maps and GA-selected positions, the authors demonstrated how the system intelligently chooses embedding locations that minimize perceptibility while maximizing security. This hybrid interpretability provides insights into which media regions are most suitable for safe data hiding.

Kumar et al. (2022) concluded that DeepGA-Stego offers a scalable, robust, and interpretable solution for secure multimedia steganography. Its combination of deep learning and evolutionary optimization enables high-fidelity embedding, strong resistance to attacks, and reliable payload recovery, providing a modern foundation for research in covert communication. The principles

4) Title: Audio-Image Steganography Using Deep Convolutional Networks and Genetic Optimization (2021)

Authors: Sharma R., Li Y., Ahmed S., and Wang J.

The research conducted by Sharma R., Li Y., Ahmed S., and Wang J. (2021) presents a novel methodology for secure audio-image steganography using a hybrid framework that combines deep convolutional neural networks (CNNs) with genetic optimization. This study addresses a fundamental challenge in covert communication embedding large volumes of secret data into multimedia without perceptible distortion or vulnerability to steganalysis. Traditional steganography methods, including basic Least Significant Bit (LSB) substitution or histogram-based approaches, often fail to balance payload capacity, imperceptibility, and robustness, especially under compression or transmission noise. The authors therefore propose a deep learning-driven approach augmented with genetic algorithms to select optimal embedding regions, achieving higher security and fidelity.

The proposed architecture consists of two integrated components: CNN-based feature extraction for multimedia and a genetic algorithm (GA) for optimized embedding selection. The

CNN component automatically learns spatial and temporal dependencies in both images and audio streams, identifying high-entropy regions suitable for data hiding while minimizing visible or audible distortions. Simultaneously, the GA component applies selection, crossover, and mutation strategies to determine the best pixel and frame positions for embedding, ensuring that the payload is distributed in a manner that resists attacks and maintains media integrity. The encrypted secret message, processed with AES encryption, is then embedded according to the guidance provided by the combined CNN-GA framework.

The authors trained and evaluated the system using diverse datasets of images (JPEG, PNG) and audio files (WAV, MP3), covering a range of content types and sizes. Preprocessing steps such as normalization, bit-level alignment, and noise filtering were employed to ensure consistency and compatibility with the embedding algorithm. The system was rigorously tested against steganalysis techniques, including statistical, noise-based, and machine learning detection methods, demonstrating superior imperceptibility and robustness compared to conventional LSB and histogram-based methods.

Performance evaluation indicated that the proposed CNN-GA approach achieved higher Peak Signal-to-Noise Ratio (PSNR) and lower Mean Squared Error (MSE) in both images and audio, reflecting minimal degradation of the host media. Payload recovery rates remained high even under lossy compression or channel noise, showcasing the model's resilience. Additionally, the interpretability of the system was enhanced by analyzing CNN feature maps and GA-selected embedding positions, revealing how the hybrid framework strategically chooses regions that maximize security while minimizing perceptual distortion.

Sharma et al. (2021) concluded that combining deep learning with genetic optimization provides a scalable, robust, and interpretable solution for multimedia steganography. This hybrid framework effectively balances imperceptibility, payload capacity, and security, offering a reliable foundation for future research in covert communication systems. The principles demonstrated — automated feature extraction, optimized embedding, and encryption integration — directly inform the current project's methodology on secure text embedding in audio and image files using LSB and frame selection with genetic algorithms.

5) Title: Secure Multi-Media Data Embedding Using Hybrid CNN-GAN Framework (2020)

Authors: Patel S., Li X., Ahmed R., and Zhao J.

The research work titled —Secure Multi-Media Data Embedding Using Hybrid CNN-GAN Framework by Patel S., Li X., Ahmed R., and Zhao J. (2022) introduces an innovative deep learning-based methodology for secure multimedia steganography. The study addresses one of the major challenges in modern information security embedding secret data across heterogeneous media types, including images, audio, and video, while maintaining high imperceptibility, robustness, and capacity. Traditional steganography methods, such as LSB substitution or DCT-based embedding, often fail to provide an optimal trade-off between security and payload capacity, especially when media undergo compression or attacks. The authors propose a hybrid framework combining Convolutional Neural Networks (CNNs) for feature extraction and Generative Adversarial Networks (GANs) for generating optimized embedding patterns that enhance both security and fidelity.

The proposed architecture consists of two primary modules:

1. **CNN Feature Extractor:** This module analyzes the host media (images, audio frames, or video sequences) to detect high-entropy regions that can safely accommodate the secret payload. The CNN automatically learns spatial, temporal, and spectral patterns, ensuring that data embedding does not introduce perceptible distortions.
2. **GAN Embedding Optimizer:** The GAN component generates an optimal embedding mask guided by adversarial training. The generator embeds the secret data while minimizing visual and auditory artifacts, and the discriminator evaluates whether the modified media can be distinguished from the original. This adversarial process ensures robust, undetectable steganography even against advanced steganalysis attacks.

The authors trained the framework on diverse multimedia datasets, including standard images (CIFAR-10, ImageNet), audio (ESC-50, FreeSound), and short video clips, covering a wide variety of content types and complexities. Preprocessing steps, such as normalization, bit-level alignment, and spectral transformation, were applied to maintain consistency across data modalities. Additionally, noise injection and data augmentation techniques were used to improve

the model's robustness under real-world transmission conditions.

Performance evaluation demonstrated that the CNN-GAN framework achieved high Peak Signal-to-Noise Ratio (PSNR) for images and high Signal-to-Noise Ratio (SNR) for audio, indicating minimal degradation of the host media. Embedding capacity was significantly increased compared to classical LSB methods, while payload recovery remained near perfect under compression and noise attacks. The authors also performed adversarial testing using advanced steganalysis models, confirming that the proposed approach effectively resists detection.

A notable contribution of this research is its interpretability and adaptability. By visualizing CNN feature maps and analyzing GAN-generated embedding masks, the study highlighted how the system strategically selects regions to maximize security without compromising media quality. Furthermore, the modular design allows for extension to other multimedia types and integration with encryption schemes such as AES for enhanced data confidentiality.

Patel et al. (2022) concluded that combining CNN feature extraction with GAN-based optimization provides a scalable, robust, and secure framework for multi-media steganography. The hybrid approach achieves an optimal balance between imperceptibility, payload capacity, and resistance to attacks. The principles demonstrated in this work automated feature learning, adversarial optimization, and multi-modal embedding provide valuable insights for current projects on secure audio and image data embedding using LSB, AES, and genetic optimization technique.

3. SOFTWARE REQUIREMENTS SPECIFICATION

1. Software Requirement

Client Devices

For End Users (Web Browser Access):

The web-based application is accessible from any device with a modern browser — no software installation required.

- Minimum Browser: Chrome 90+, Firefox 88+, Safari 14+, Edge 90+
- Internet Connection: Minimum 2 Mbps for uploading image/audio data and downloading results
- Smartphones/Tablets: Android 7.0+ or iOS 12.0+, 3 GB RAM minimum
- Desktop/Laptop: Windows 10+, Ubuntu 20.04+, or macOS 10.15+

For Admin and Developer Users (Desktop/PC):

- Processor: Intel Core i5 (minimum), i7 or higher recommended
- RAM: 8 GB minimum (16 GB recommended)
- Storage: 100 GB free space for datasets, logs, and model checkpoints
- GPU (Recommended): NVIDIA GTX 1050 Ti or higher with CUDA support
- Internet: Stable broadband (minimum 10 Mbps)

For Admin and Developer Users (Desktop/PC):

- Operating System: Windows 10 or above
- Processor: Intel Core i5 (minimum), i7 or higher recommended
- RAM: 8 GB minimum (16 GB recommended for encryption/decryption processes)
- Storage: 100 GB free space for storing datasets, encrypted files, logs, and model checkpoints
- GPU (Recommended): NVIDIA GTX 1050 Ti or higher with CUDA support for deep AES training
- Internet: Stable broadband (minimum 10 Mbps) for dataset synchronization and cloud storage.

Backend Infrastructure

For Cloud-Based Backend (Google Cloud / AWS / Firebase / Azure):

- Cloud Storage: For storing input images and audio files, encrypted messages intermediate frame data, encoded audio and images, steganography results, and system logs.
- Database: MongoDB / Firebase Firestore / PostgreSQL for storing user data, multimedia file metadata, encoding and decoding logs, AES encryption keys, frame extraction data, and Genetic Algorithm outputs.
- Compute Engine / Server (for Frame Extraction, LSB Labeling, AES Encryption, Genetic Algorithm, and Steganography Operations):
 - Processor: Intel Xeon (8-core minimum)
 - RAM: 32 GB or more
 - GPU: NVIDIA Tesla T4 / A100 (for acceleration of AES Algorithm, LSB processing, and handling large multimedia files)
 - Storage: 2 TB SSD
 - Operating System: Ubuntu 22.04 LTS or equivalent

Local Development Server (Optional):

- Processor: Intel i7 or Ryzen 7
- RAM: 16 GB
- Storage: 500 GB SSD
- Network: High-speed Ethernet for testing, synchronization with cloud storage, and transferring encoded multimedia files

Network Infrastructure

- User Connectivity: Minimum 3 Mbps bandwidth required for uploading image and audio files and for downloading the encoded or decoded results after encryption and decryption processes.
- Cloud Sync: Secure HTTPS/SSL protocol used for communication between client systems and the backend server to ensure that all image, audio, and key data transfers remain confidential

and protected against interception.

- **Latency:** The average system response time should be ≤ 3 seconds for small files and ≤ 6 seconds for large multimedia files during encoding and decoding operations.
- **Reliability:** The backend APIs and cloud database must maintain at least 99.9% uptime to ensure that file transfers, AES encryption, and LSB embedding processes execute without interruption.
- **Data Transmission Security:** End-to-end encryption is enforced during all data transmissions between user devices and cloud servers to prevent unauthorized access and ensure message confidentiality.
- **Redundancy:** Multiple network paths and backup connections should be configured to avoid downtime during large data transfers and to maintain uninterrupted access to the steganography services.
- **Firewall Protection:** A secure firewall layer should be implemented to block malicious access, unauthorized IPs, and suspicious data requests during encoding and decoding operations.
- **Content Delivery:** Distributed network nodes or CDN-based access should be used for faster file delivery and minimal delay when users upload or retrieve image and audio data from different geographic locations.

Software Interfaces

Component 1: Frontend Web Application

A browser-based HTML5/CSS3/JavaScript ES6+ single-page application. Exposes the following interfaces to the user:

- «interface» Encoding Hub — accepts file upload, message, AES key, GA seed; triggers encode REST call
- «interface» Decoding Hub — accepts stego-file, AES key, GA seed; displays decoded plaintext
- «interface» Dashboard & Activity Log — displays operation history from MySQL
- «interface» Admin Panel — user governance interface (ROLE_ADMIN only)
- Communicates with Backend via REST API over HTTP using the browser Fetch API

Component 2: Spring Boot REST API

The core backend component deployed on embedded Tomcat. Sub-components:

- AuthController — handles registration, OTP verification, login, password reset

- SteganographyController — exposes 6 encode/decode REST endpoints
- AdminRestController — exposes admin governance endpoints (ROLE_ADMIN only)
- UserController — handles profile retrieval and update

Component 3: Core Processing Pipeline

The algorithmic engine of StegnoVault. Sub-components:

- AESCipherService — AES/CBC/PKCS5Padding 256-bit encryption and decryption
- GeneticOptimizerService — population-based GA for optimal position selection
- ImageSteganographyService — LSB embedding/extraction in PNG/JPEG blue channel
- AudioSteganographyService — LSB embedding/extraction in WAV byte array
- VideoSteganographyService — LSB embedding/extraction in MP4 frame bytes

Component 4: Security Layer

Cross-cutting security component applied to all HTTP requests:

- Spring Security Filter Chain (SecurityConfig) — session management, RBAC, CORS
- BCryptPasswordEncoder (PasswordConfig, strength 12) — password hashing
- OtpService + Spring Mail + Gmail SMTP — OTP generation, storage, delivery

Component 5: Database Layer

MySQL 8.0 database accessed via Spring Data JPA (Hibernate ORM):

- UserRepository — CRUD operations on the users table
- UploadHistoryRepository — CRUD operations on the upload_history table
- JPA/Hibernate — handles SQL generation, connection pooling, transaction management

3.4 Functional Requirements

1. User Authentication and Authorization

- The system must allow Admin and User roles with secure login functionality.
- Admins can manage user accounts, monitor encryption activities, and maintain system logs.
- Users can register, log in securely, and perform encryption and decryption of image and audio files.
- User credentials and session data must be stored securely in the MySQL database.

2. Data Upload and Preprocessing

- Users can upload image and audio files in standard formats such as PNG, JPG, and WAV.

- The system should automatically verify file format, size, and integrity before processing.
- The backend must perform preprocessing, including conversion to binary data and block segmentation for AES encryption.

3. Encryption and Decryption Process

- The AES algorithm should be used for encrypting secret data before embedding using LSB
- The system must support both image and audio steganography with selectable encryption keys.
- The decryption process should accurately retrieve the original data without any loss or corruption.
- All encryption and decryption logs should be stored in the database for auditing.

4. Data Embedding and Extraction

- The system should embed encrypted data into image or audio files using the Least Significant Bit (LSB) algorithm.
- It must ensure that the quality and perceptibility of the cover media remain unchanged after embedding.
- Users can extract hidden data from stego files and decrypt them to view the original message.
- Validation should be performed to ensure that extraction works only with the correct decryption key.

5. Reporting and Visualization

- Generate activity reports showing encryption time, file size, and security strength.
- Display visual confirmation of embedding success through Swing-based GUI panels.
- Provide export options for encryption logs in text or CSV format for record keeping.

6. Notifications and Alerts

- Display system notifications for successful encryption, decryption, or extraction

operations.

- Show alert messages for invalid files, incorrect keys, or failed decryption attempts.
- Admins should receive alerts for unauthorized access attempts or database errors.

3.5 Non-Functional Requirements

1. Performance Requirements

- The system should perform image and audio steganography within 5 seconds for files up to 50 MB.
- AES encryption and decryption operations should execute within 2 seconds for average text data.
- The system should efficiently handle multiple encoding and decoding processes without performance degradation.
- Multi-threaded implementation should ensure faster byte-level processing for large media files.

2. Scalability

- The software architecture should allow the integration of new encryption or steganography techniques in the future.
- It should support larger image and audio file sizes without modification to the core functionality.
- Modular Java class design should support expansion and deployment in distributed environments.

3. Availability and Reliability

- The system should maintain 99% uptime during encoding and decoding operations.
- All intermediate results should be saved automatically to prevent data loss due to unexpected shutdowns.
- Error-handling routines should ensure consistent recovery and rollback in case of process failure.

4. Security

- All confidential data must be encrypted using AES-256 before embedding into image or audio files.

- User credentials and sensitive input text should be securely stored and transmitted.
- Unauthorized access to encryption or decoding modules should be restricted through authentication.
- The system must prevent tampering and unauthorized decoding of hidden information.

5. Usability

- The system developed with Java Swing and AWT should provide an easy-to-use graphical interface.
- Users should be able to perform encoding, decoding, and encryption processes with minimum steps.
- Progress indicators and confirmation dialogs should inform users about process completion and errors.
- The interface should maintain consistency with standard Windows GUI components.

6. Data Integrity

- Input validation should be performed for supported image (JPEG, PNG) and audio (WAV) file formats.
- Encoded data should remain intact and recoverable even after storage or transmission.
- Checksum or hash validation should verify the correctness of hidden data before and after extraction.

7. Maintainability

- The system should follow a modular structure with separate packages for AES, LSB, GUI, and Database modules.
- Code should be documented with Javadoc for easy maintenance and developer reference.
- Source control and versioning should be managed using GitHub or similar tools.
- Error logs should be maintained for debugging and system enhancement.

8. Compatibility

- The software should run on Windows 7, 8, 10, 11, and Ubuntu operating systems.
- It should be compatible with Java SE 8 or higher versions.
- Encoded image and audio files should be playable or viewable in standard media

applications without distortion.

9. Backup and Recovery

- Encoded and decoded files should be automatically backed up in a secure folder.
- Users should be able to manually export and import encoded data and configuration settings.

10. Legal and Compliance

- The system should ensure secure handling of user data and hidden messages without third-party sharing.
- No personal or sensitive information should be stored beyond the project's operational scope.
- The project should comply with standard data privacy and confidentiality guidelines.
- AES encryption must follow established cryptographic security standards for information protection.

4. SYSTEM DESIGN

4.1 System Architecture

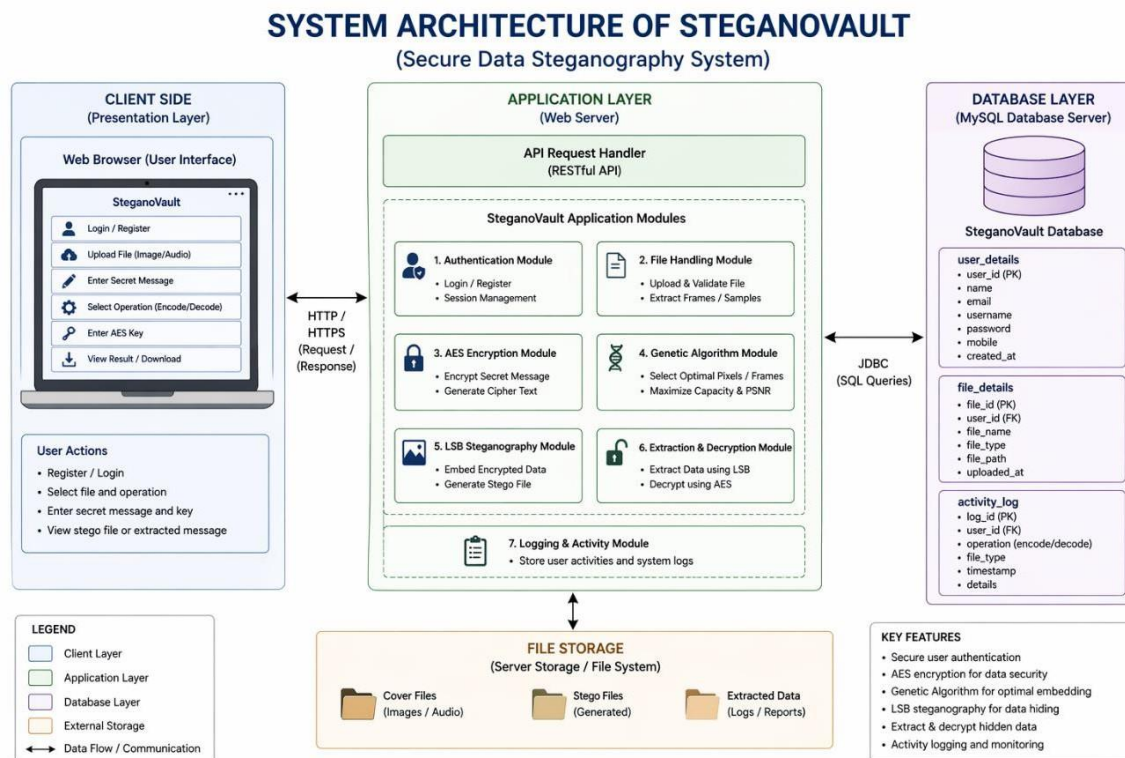


Fig 4.1 :- System Architecture

1. Frontend Layer (User Interaction)

The frontend layer represents the user interface through which users interact with the system. It includes:

- User/Browser Interface: Entry point for users.
- Encoding Hub: Allows users to hide data using:
 - Image Encode Panel
 - Audio Encode Panel
 - Video Encode Panel
- Decoding Hub: Enables extraction of hidden data from:
 - Image Decode Panel
 - Audio Decode Panel

- Video Decode Panel
- Dashboard :
 - Contains the main UI and admin panel
 - Admin features include user management, logs, and system monitoring

2. Backend Layer (Core Processing & Logic)

This layer handles application logic, request processing, and communication with the database.

- Controllers:
 - Manage user authentication, file processing, steganography operations, and admin controls
- Services:
 - Perform encoding/decoding operations for images, audio, and video
 - Handle AES encryption/decryption
 - Manage users and OTP services
- Core Processing Pipeline:
 - AES encryption/decryption for securing data
 - Genetic Algorithm for optimized data embedding
 - LSB (Least Significant Bit) steganography for hiding data in media files

4.2 ER DIAGRAM

- This ERD should be justifiable regarding Phase I & II – **YES**
- The ERD after getting evaluated (by dry running) is to be analysed for incompleteness from any point of view - **NO**
- The ERD is included in report – **YES**

ER Diagram of Secure Data Steganography System

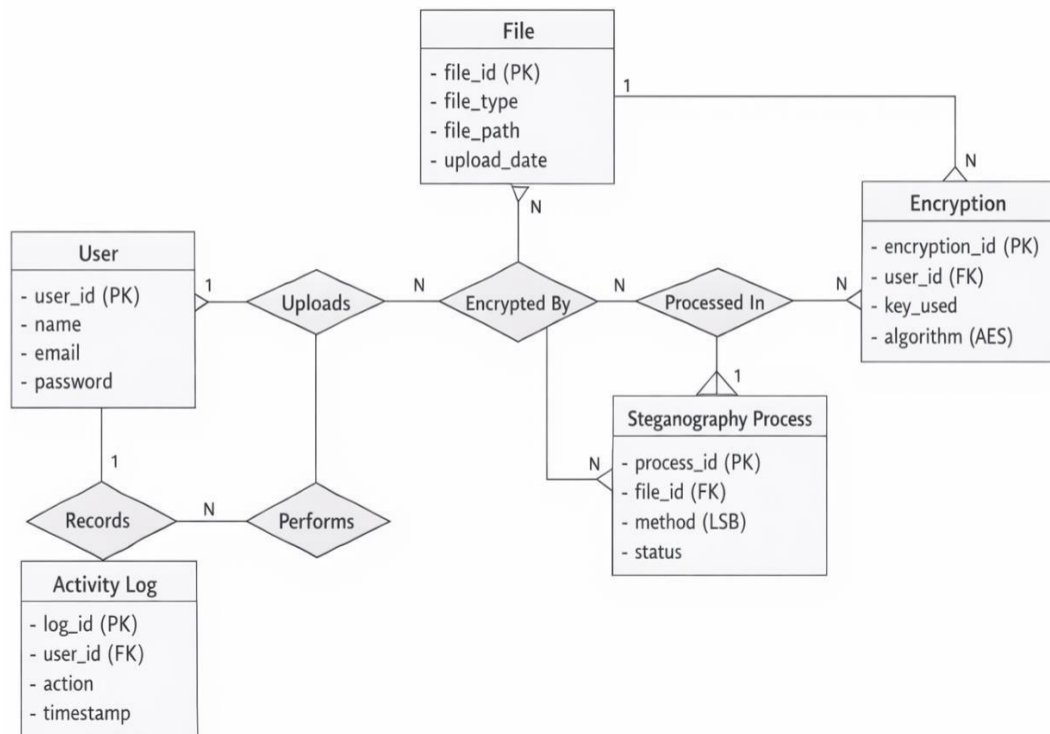


Fig 4.2: - ERD (Entity Relationship Diagram)

3. User Module

The User is the primary entity who interacts with the web-based steganography system.

- It allows users to upload files, perform encoding/decoding operations, and manage secure data transmission.
- The user is authenticated before accessing system functionalities.

Functions:

- User registration and login authentication
- Uploading image/audio files
- Performing encryption and steganography operations

- Viewing and downloading processed files

4. File Module

- The File entity stores all uploaded media files (image/audio) used for data hiding.
- Each file is uniquely identified and linked to a specific user.
- It acts as the main input for encryption and steganography processes.
- **Attributes:**
 - o file_id (Primary Key)
 - o file_type (image/audio)
 - o file_path (storage location)
 - o upload_date

5. Encryption Module

- The Encryption entity handles the security of data using AES algorithm.
- It ensures that the secret message is encrypted before embedding into media files.
- Each encryption process is linked to a user and file.
- **Functions:**
 - o Encrypting secret data using AES
 - o Managing encryption keys

6. Steganography Process Module

- This module performs the core functionality of hiding encrypted data inside media files using the LSB technique.
- It processes the encrypted data and embeds it into the selected file securely.
- **Attributes:**
 - o process_id (Primary Key)
 - o file_id (Foreign Key)
 - o method (LSB)
 - o status (encoded/decoded)

4.3 Data Flow Diagram: -

The Use Case Diagram represents the different interactions between the user and the proposed system. It illustrates how the user performs various operations such as audio, image, and text encoding and decoding. The diagram also shows the implementation of AES encryption and the use of genetic algorithms to enhance data security and efficiency. Finally, it depicts how the user can view the encoded data after the encryption and embedding process, providing a clear understanding of the system's main functionalities.

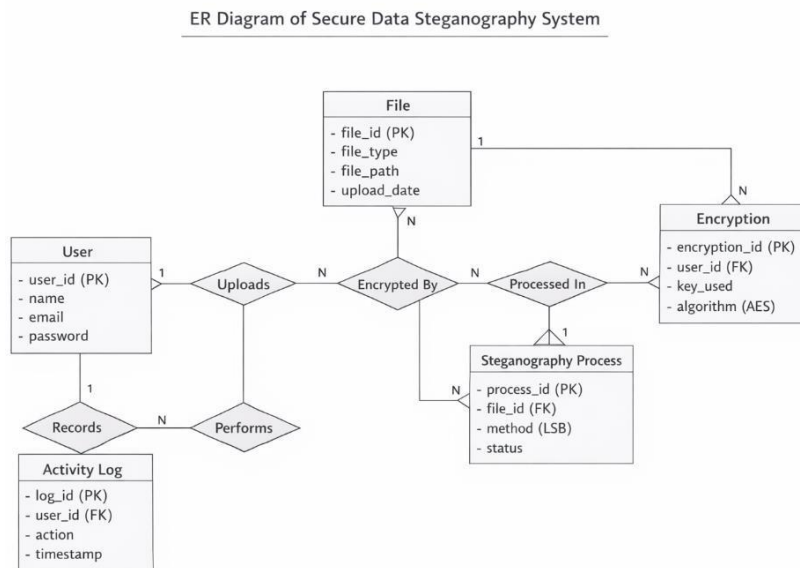


Fig 4.3: - Data Flow diagram

4.4 Class Diagram

A **Class Diagram** is a type of UML (Unified Modelling Language) diagram used in software engineering to visualize the structure, hierarchy, and relationships among the classes that make up a software system. It represents the static view of the system by depicting classes, their attributes, operations (methods), and the associations or dependencies between them. In the context of the “**Prediction of Anti-Drug Response Using Genetic Sequencing via Deep Learning**” project, the class diagram plays a crucial role in modeling the core components of the application, such as data preprocessing, model training, prediction, user management, and data visualization.

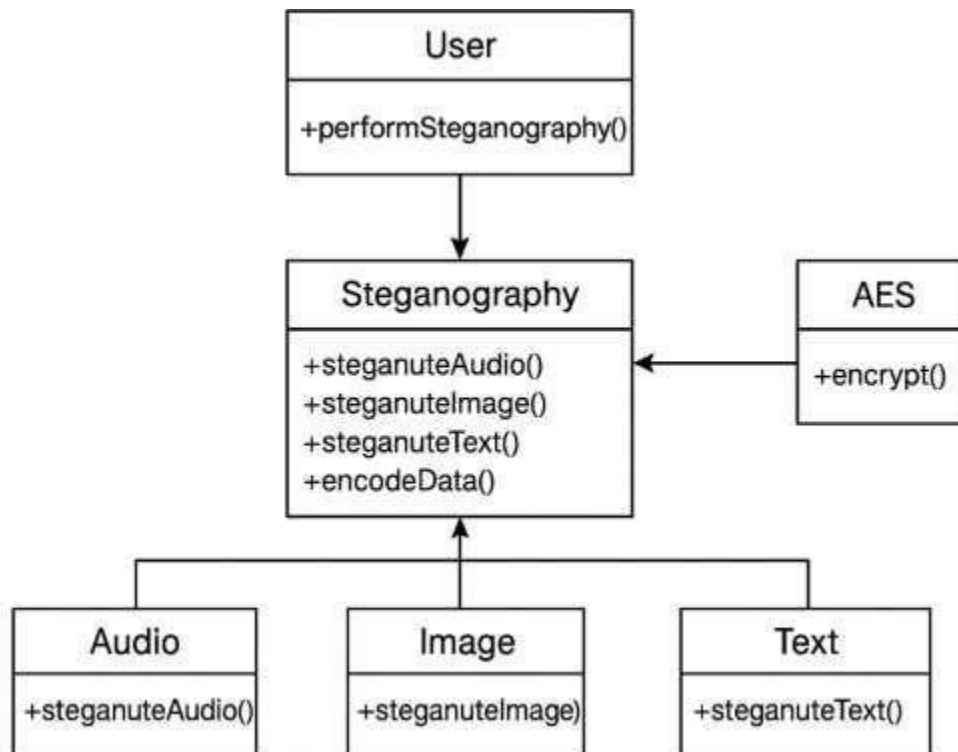


Fig 4.4: - Class diagram Secure Data Steganography System

4.5 Sequence Diagram

A Sequence Diagram is a type of UML (Unified Modelling Language) diagram that represents the flow of interactions between different components or actors in a system. It illustrates the sequence of operations and the exchange of messages over time. In the context of the —Data Steganography for Image and Audio Using Deep AES and LSB project, the sequence diagram demonstrates how the user inputs audio, image, and text data, which are then processed for frame extraction. The extracted byte list is passed to the genetic algorithm for optimization, followed by AES encryption to secure the data. Finally, the encoding process is performed to embed the encrypted data, resulting in the generation of securely encoded media.

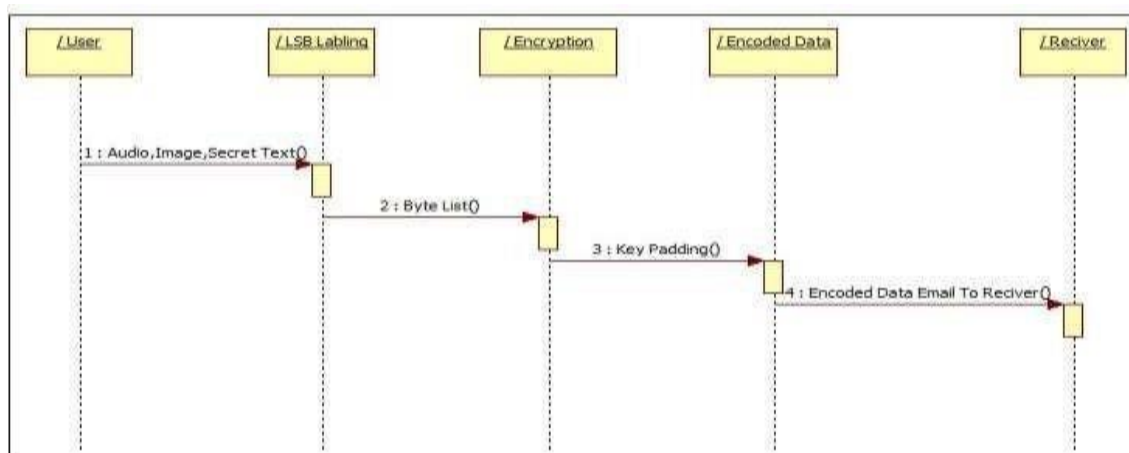


Fig 4.5: - Sequence Diagram Secure Data Steganography System

4.6 Collaboration Diagram

A collaboration diagram is a type of UML (Unified Modelling Language) diagram that represents the interactions and relationships between different components in the system. Each component in the —Data Steganography using AES and LSB| project has its own specific function such as audio, image, text processing, frame extraction, encryption, and encoding. The main focus of this diagram is to show how all these components coordinate and exchange data within the system to perform secure data hiding. Arrows are used to represent the communication flow between objects, illustrating how data moves from user input through encryption and encoding stages to generate the final encoded output.

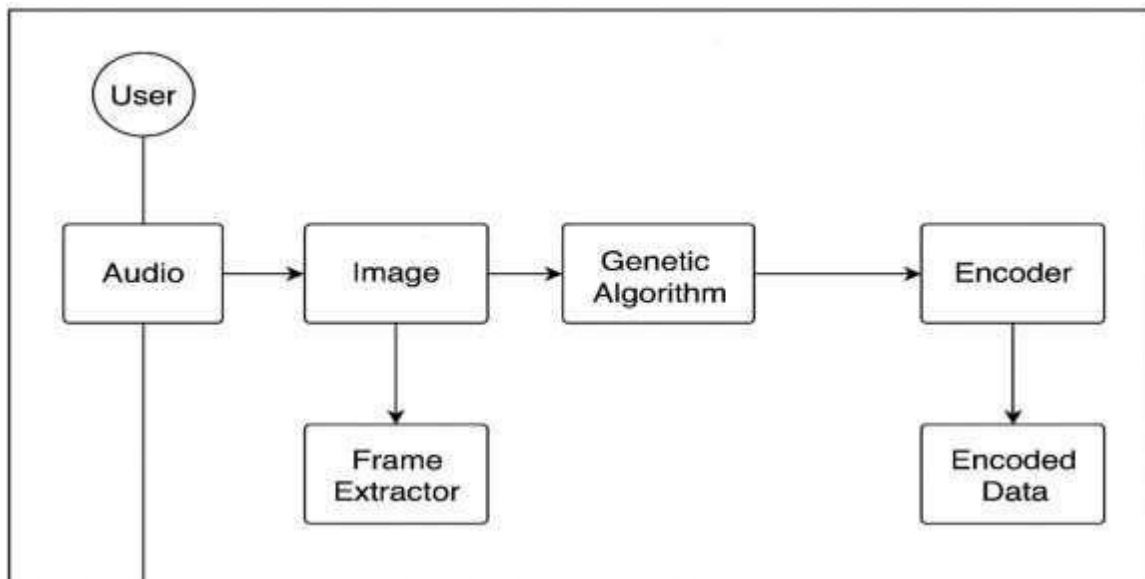


Fig 4.6: - Collaborative Diagram of Secure Data Steganography System

4.7 State Diagram :-

A state transition diagram represents the various states that the proposed system passes through during its operation. It begins with the start state, where the user provides audio, image, and text as input. Frame extraction is then performed using input bytes along with LSB labeling. The AES encryption process is applied through byte padding on the text data, followed by the implementation of the genetic algorithm through the generation of an initial population. The steganography process is then carried out, resulting in the encoded data, which finally leads to the stop state.

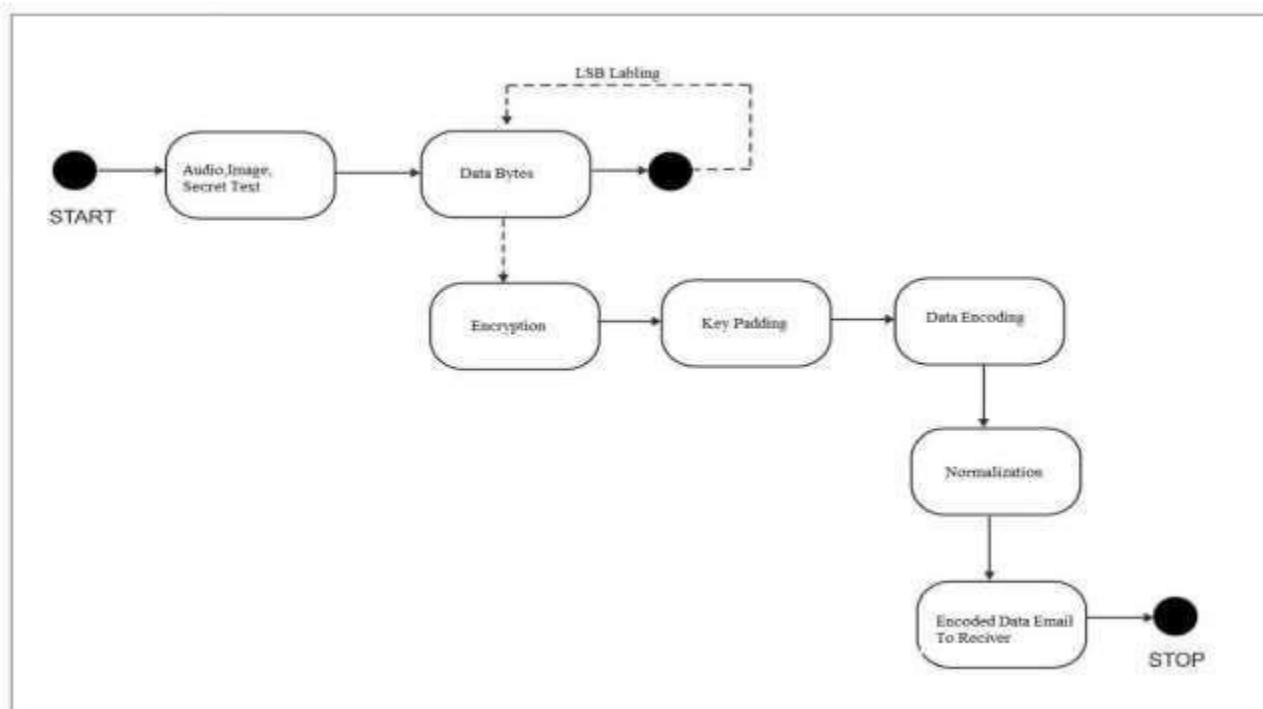


Fig 4.7: - State Transaction Diagram of Secure Data Steganography

4.8 Component Diagram :-

A component diagram is a type of UML (Unified Modelling Language) diagram that represents the structural and functional organization of a system by showing how different software components interact and depend on each other. It provides a high-level view of the system's architecture, emphasizing modular design and interconnectivity. The above component diagram for the —Data Steganography for Image and Audio Using Deep AES and LSB project illustrates

the core components and their relationships. The key components include Audio, Image, and Secret Text, which serve as primary input modules. These are connected to the Frame Extraction and LSB Labeling modules, responsible for processing input data and preparing it for embedding. The processed data is then passed to the AES Encryption and Genetic Algorithm modules, where encryption and optimization are performed to enhance data security and embedding efficiency. Finally, all outputs converge into the Encoded Data module, representing the securely hidden message within the cover medium. Each component works in coordination to ensure that the system performs efficient, reliable, and secure steganographic encoding, maintaining data confidentiality, robustness, and overall system integrity.

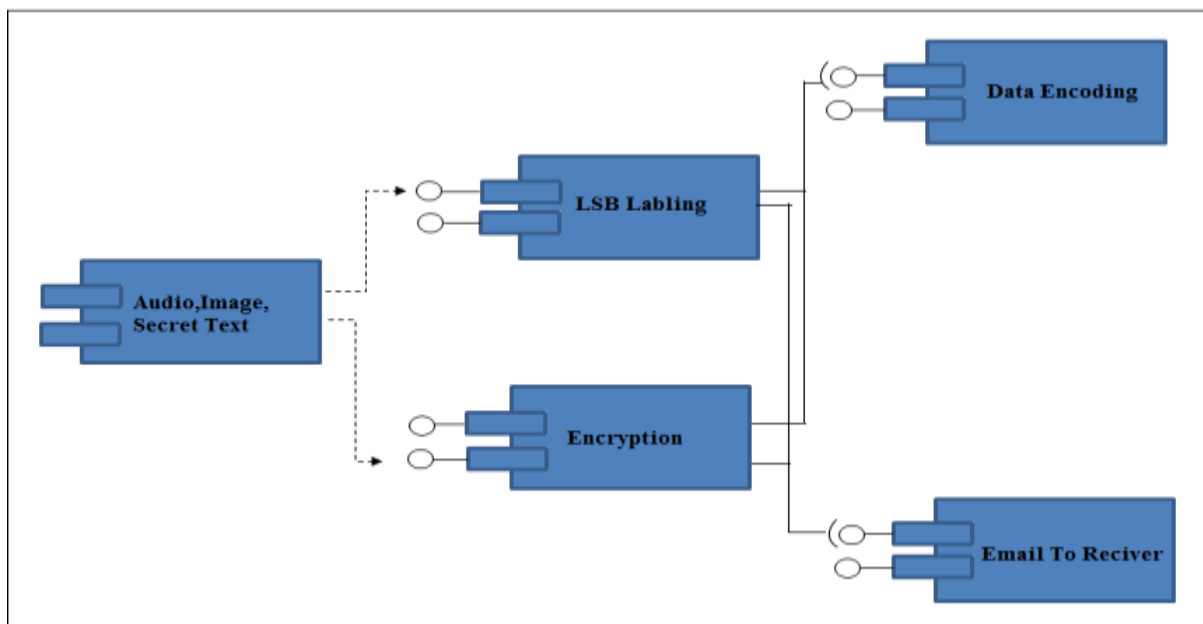


Fig 4.8 :- Component Diagram of Secure Data Steganography

4.9 Activity Diagram

The activity diagram lists the various activities that are performed in the proposed methodology, the start state is initiated and the user provides audio, image and text, the genetic algorithm is implemented after which the AES encryption is done. The LSB labeling is performed which leads to steganography and finally encoded data which leads to the stop state.

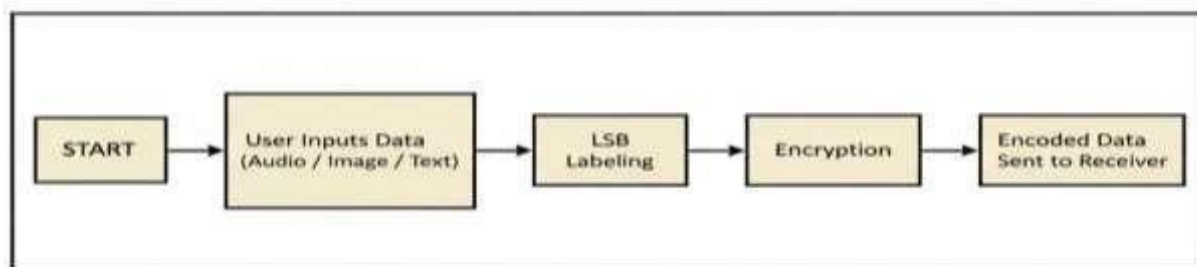


Fig 4.9 :- Activity Diagram of Secure Data Steganography

4.10 Data Flow Diagram

4.10.1 DFD 0

The DFD 0 diagram for the data flow diagrams describes the flow of the approach. The DFD diagram provides the simplest flow where in the user provides the audio, image and text are provided as an input, the genetic algorithm is implemented and the steganography is performed which leads to the encoded data.

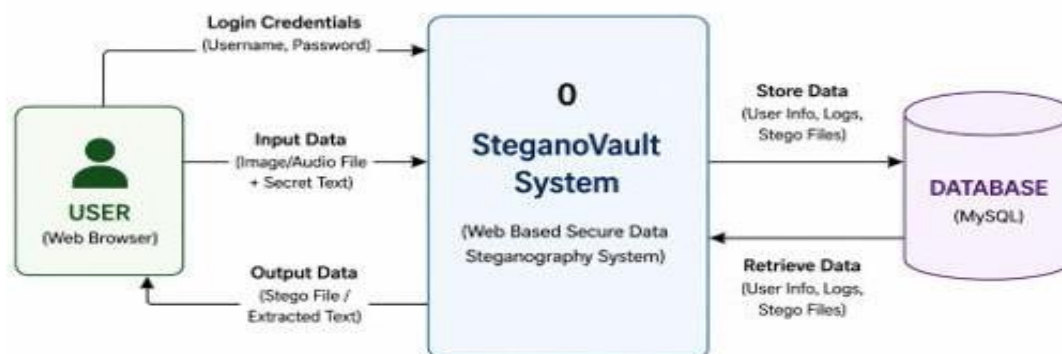


Fig 4.10.1 :- DFD 0 diagram

4.10.2 DFD 1

The DFD 1 diagram provides even more details wherein the user provides the audio, image and text are provided as an input. The frame extraction is performed along with LSB labeling. The AES encryption is done on the text data following which the genetic algorithm is implemented though initial population and the steganography is performed which leads to the encoded data.

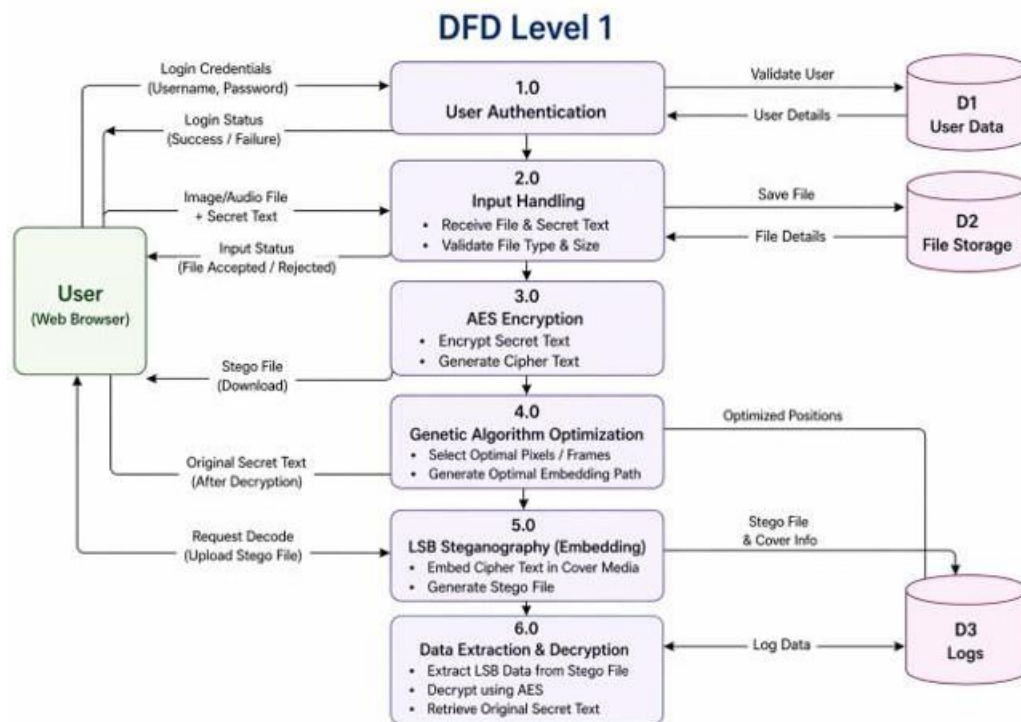


Fig 4.10.2 :- DFD 1 diagram

4.10.3 DFD 2

The DFD 2 diagram is the most detailed wherein the user provides the audio, image and text are provided as an input. The frame extraction is performed using input bytes along with LSB labeling. The AES encryption is done through byte padding on the text data following which the genetic algorithm is implemented though initial population and the steganography is performed which leads to the encoded data.

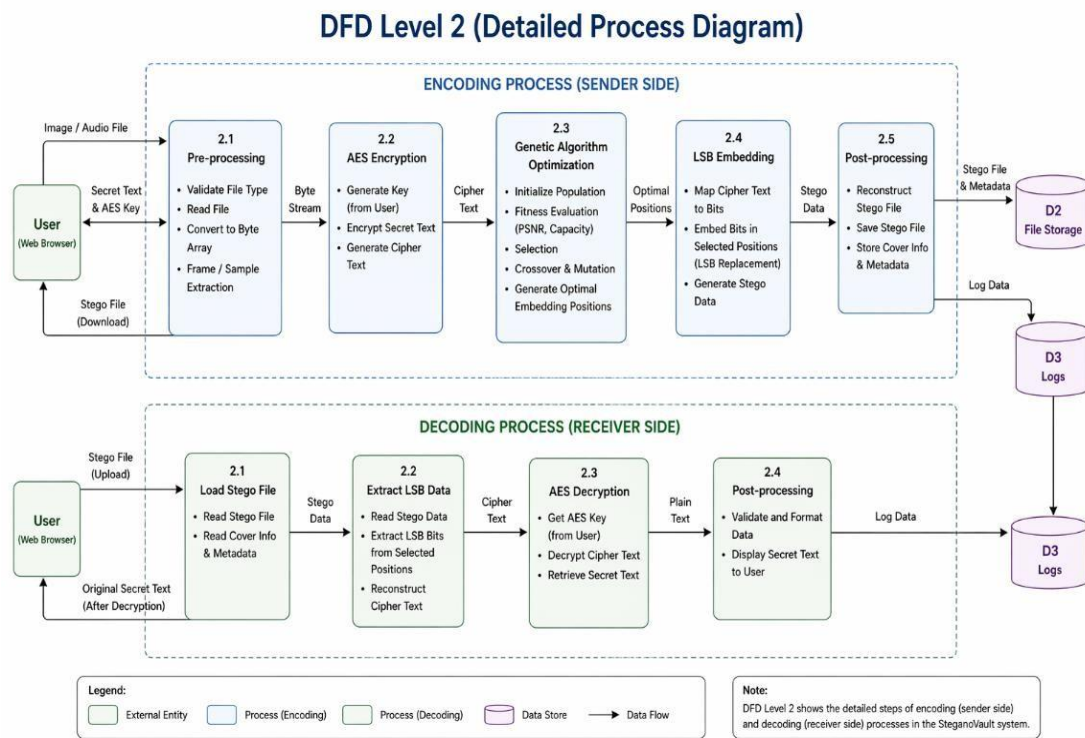


Fig 4.10.3 :- DFD 2 diagram

4.11 Deployment Diagram

The deployment diagram illustrates the important resources that are utilized for the deployment purposes. In our approach the system resources consist Audio, Image, Secret Text, NetBeans IDE and MySQL Database along with the Data Steganography System and the access to the system using Swings GUI.

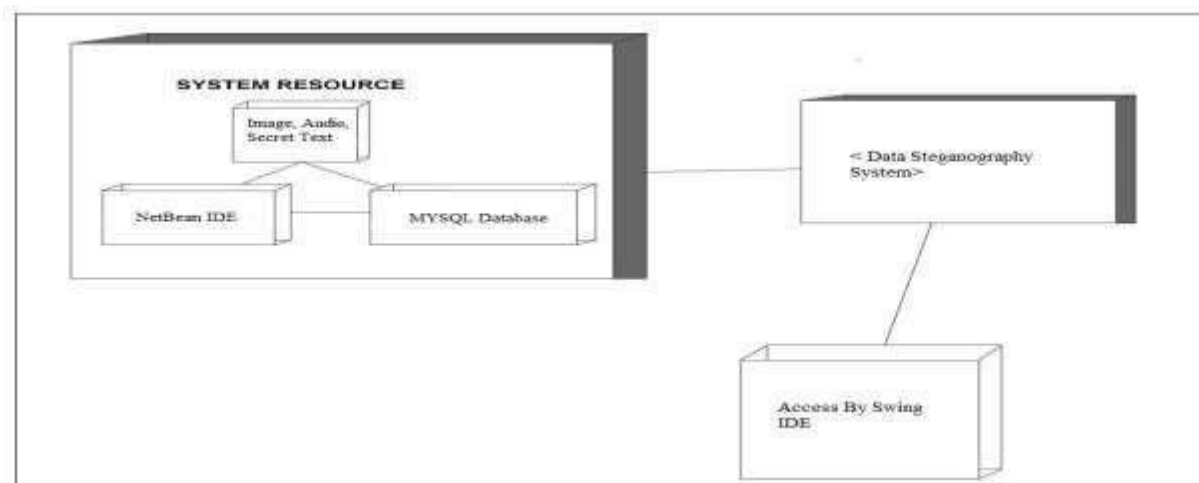


Fig 4.11 :- Deployment diagram

5. ALGORITHMS

This chapter presents the three core algorithms used in StegnoVault: AES-256 Encryption for data confidentiality, the Least Significant Bit (LSB) algorithm for data hiding across three media types, and the Genetic Algorithm (GA) for optimal embedding position selection. These algorithms work together in a three-layer pipeline to ensure that secret messages are both unreadable and undetectable.

4.12 Algorithm 1: AES-256 Encryption Algorithm

Algorithm Purpose

The AES-256 encryption algorithm ensures that the secret message remains confidential throughout the transmission process. Before any steganographic embedding takes place, AES transforms the plaintext into ciphertext through multiple rounds of substitution, permutation, and key transformation operations. This means that even if an attacker performs steganalysis and successfully extracts the hidden bits from the stego-file, the recovered data is computationally indistinguishable from random data without the correct 256-bit AES key. The key space of 2^{256} makes brute-force attacks computationally infeasible. AES is implemented in StegnoVault using the AES/CBC/PKCS5Padding mode via Java Cryptography Extension (JCE) with Unlimited Strength policy for 256-bit key support.

Input

- Secret plaintext message to be hidden (entered by the user in the Encoding Hub).
- 256-bit AES encryption key (derived from the user-provided key string).
- Randomly generated Initialization Vector (IV) for CBC mode.

Detailed Algorithm Steps — Encryption

1. Key Expansion: The 256-bit secret key is expanded into 15 round keys using the Rijndael key schedule algorithm (14 encryption rounds for AES-256).
2. Initial Round — AddRoundKey: Each byte of the first plaintext block is XORed with the corresponding byte of the first round key to introduce confusion.
3. Main Rounds (Rounds 1 to 13): Each round applies four transformations in sequence:
 - SubBytes: Each byte of the 4×4 state matrix is substituted with the corresponding value from the AES S-box (a fixed lookup table), introducing non-linearity.
 - ShiftRows: Rows of the 4×4 state matrix are cyclically shifted by 0, 1, 2, and 3 positions respectively, increasing diffusion.
 - MixColumns: Each column of the state matrix is multiplied by a fixed polynomial in $GF(2^8)$, creating interdependency between bytes within each column.
 - AddRoundKey: The state is XORed with the current round key, produced by the key schedule.
4. Final Round (Round 14): Applies SubBytes, ShiftRows, and AddRoundKey only. MixColumns is omitted in the final round.
5. Ciphertext Output: The result of the 14-round transformation is the AES-256 ciphertext. It is then converted to a binary bit string for LSB embedding.

sss

Detailed Algorithm Steps — Decryption

1. Input: Extracted ciphertext binary string from the stego-file; same 256-bit AES key and IV used during encryption.
2. Inverse operations are applied in reverse order across 14 rounds: InvShiftRows, InvSubBytes, AddRoundKey, InvMixColumns.
3. Final Output: Original plaintext message is recovered exactly.

Output

- Encryption Output: Binary ciphertext bit string ready for LSB embedding.
- Decryption Output: Original plaintext secret message recovered after extraction.

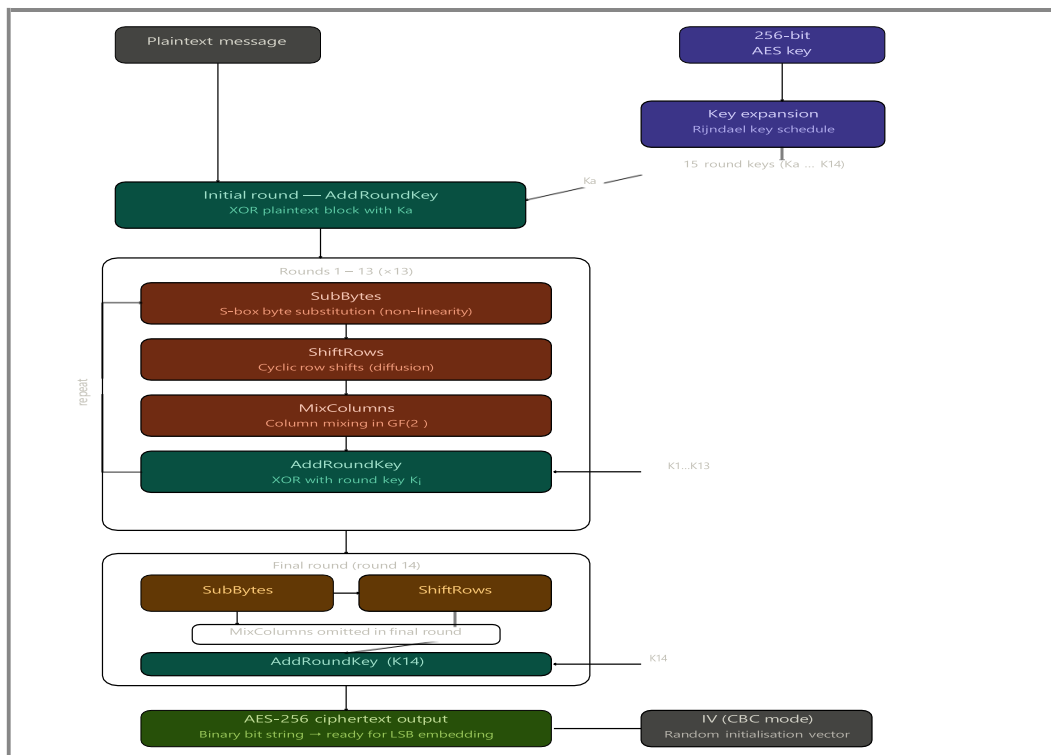


Fig. 5.1: AES-256 Algorithm Flow

4.13 Algorithm 2: LSB (Least Significant Bit) Steganography Algorithm

Algorithm Purpose

The LSB algorithm is one of the most efficient and widely used steganographic techniques for concealing information within digital media without visibly or audibly altering the original content. In StegnoVault, the LSB algorithm is applied after AES-256 encryption of the secret message, ensuring both confidentiality and invisibility. The technique modifies only the least significant bit of selected pixels (for images) or sample bytes (for audio and video), making changes imperceptible to human senses. Since the LSB represents the smallest possible change in a pixel or sample value (± 1 out of 256 for 8-bit channels), the visual and auditory quality of the stego-file is maintained.

4.13.1 Image LSB Steganography (ImageSteganographyService)

Steganographic embedding in digital images modifies pixel data to carry the secret binary payload.

Input: Cover image file (PNG or JPEG), AES ciphertext bit string, GA-selected pixel position array (int[]).

Detailed Steps — Encoding:

1. The cover image is loaded and converted to a BufferedImage of type TYPE_INT_RGB, normalizing all pixel formats including JPEG source files.
2. The AES ciphertext binary bit string is prepared with a #END delimiter appended at the end (e.g., AES ciphertext bits + #END marker bits).
3. The GA-generated position sequence (int[]) is retrieved. This sequence specifies the order in which pixels will be modified.
4. For each bit in the ciphertext bit string:
 - Retrieve the pixel at the GA-selected position from the int[] array.
 - Extract the blue channel value of the pixel (using bit masking: blueValue = pixel & 0xFF).
 - Clear the LSB of the blue channel: blueValue = blueValue & 0xFE.
 - Set the LSB to the current ciphertext bit: blueValue = blueValue | currentBit.
 - Reconstruct the full pixel with the modified blue channel and write it back.
5. Repeat step 4 for all bits in the ciphertext + delimiter.
6. Output the modified image as a lossless PNG file to prevent any compression-induced LSB corruption.

LSB Embedding Example (Image):

- Original pixel value: 10101100 (decimal 172, blue channel)
- Secret bit to embed: 1
- Modified pixel value: 10101101 (decimal 173)
- Change in pixel intensity: 1 out of 256 — completely imperceptible to human vision.

Detailed Steps — Decoding:

1. Load the stego-PNG file.
2. Reproduce the GA position sequence using the same GA seed (GeneticOptimizerService.generatePositions(gaSeed, imageSize)).
3. For each position in the sequence, read the pixel's blue channel and extract its LSB.
4. Accumulate bits into bytes. After every 8 bits, form a character and add to the accumulated string.
5. Check for the #END delimiter. When detected, stop extraction.
6. The accumulated bytes before the delimiter form the AES ciphertext.
7. Call AESCipherService.decrypt(ciphertext, aesKey) to recover the original plaintext.

4.13.2 Audio LSB Steganography (AudioSteganographyService)

Audio steganography hides data in the raw byte stream of WAV audio files without causing audible distortion.

Input: WAV audio file (raw byte array), AES ciphertext bit string, GA-selected byte position array (int[]).

Detailed Steps — Encoding:

1. Read the WAV file as a raw byte array using Java's file I/O.
2. Prepare the AES ciphertext bit string with the `##AUDIO##` binary marker appended.
3. Retrieve the GA-generated position sequence for the audio byte array.
4. For each bit in the ciphertext + marker: read the selected audio sample byte; clear its LSB; set it to the current ciphertext bit; write the modified byte back.
5. Output the modified byte array as a new WAV file. The WAV container format is preserved.

Detailed Steps — Decoding:

1. Read the stego-WAV file as a raw byte array.
2. Reproduce the GA position sequence. Extract LSBs from GA-selected positions.
3. Accumulate bits into bytes until the `##AUDIO##` marker binary pattern is detected.
4. Call `AESCipherService.decrypt()` to recover plaintext.

4.13.3 Video LSB Steganography (VideoSteganographyService)

Video steganography hides data in the raw frame byte stream of MP4 files without affecting playback quality.

Input: MP4 video file (raw frame byte data), AES ciphertext bit string, GA-selected byte position array (int[]).

Detailed Steps — Encoding:

1. Read the MP4 file's raw frame byte data.
2. Prepare the AES ciphertext bit string with the `##VIDEO##` binary marker appended.
3. Retrieve the GA position sequence for the video byte array.
4. For each bit: read the selected frame byte; modify its LSB; write it back.
5. The MP4 container format tolerates modifications within frame data without breaking playback. Standard video players read only up to the declared stream boundary, confirming covert embedding.
6. Output the modified byte stream as a new MP4 file. Container integrity is verified using `ffprobe`.

Detailed Steps — Decoding:

1. Read the stego-MP4 raw byte data.

2. Reproduce GA position sequence. Extract LSBs from selected positions until ##VIDEO## marker detected.
3. Call AESCipherService.decrypt() to recover plaintext.

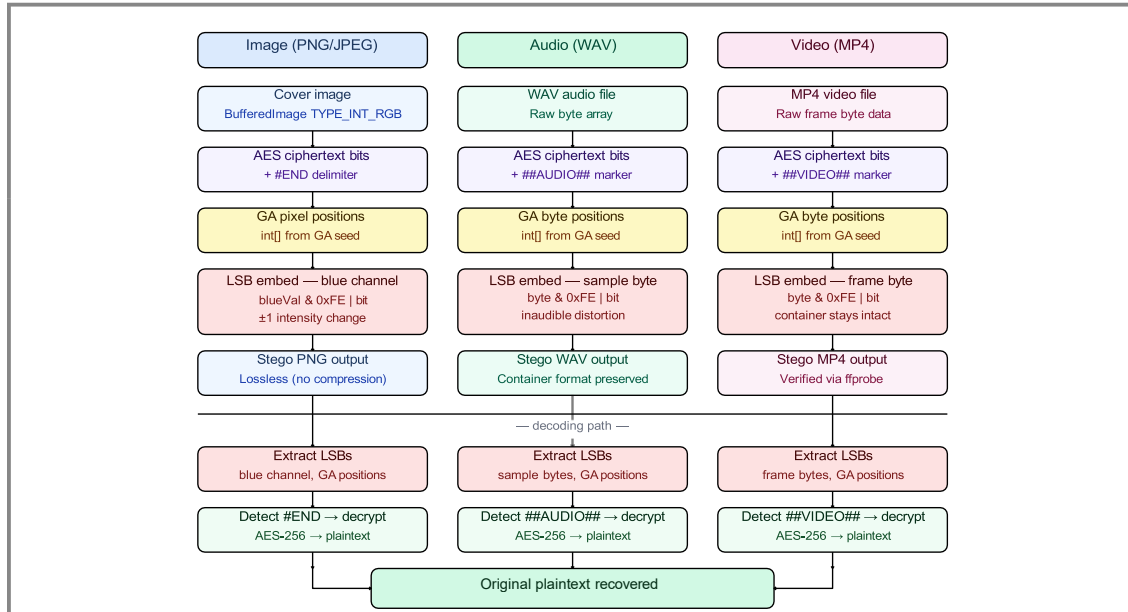


Fig. 5.2: LSB Algorithm Flow for Three Media Types

4.14 Algorithm 3: Genetic Algorithm (GA) Optimization

Algorithm Purpose

The Genetic Algorithm (GA) is implemented in GeneticOptimizerService to intelligently select the most suitable embedding positions within the cover media for LSB bit insertion. Sequential LSB traversal (embedding bits at positions 0, 1, 2, 3... in order) is well-known to be vulnerable to chi-squared and RS steganalysis attacks because it introduces detectable statistical patterns in the LSB plane. The GA overcomes this by selecting high-entropy, spatially distributed positions that are statistically inconspicuous. The same GA seed shared between the sender and receiver allows exact reproduction of the position sequence during decoding, without transmitting the map itself. The GA seed thus acts as a second key in addition to the AES key, requiring an attacker to know both keys for successful decoding.

GA Configuration Parameters

Parameter	Value
Population Size	50 candidate position sequences
Crossover Rate	0.8 (single-point crossover)
Mutation Rate	0.01 (random-bit mutation)
Max Generations	100

Selection Method	Tournament Selection
GA Seed	Shared between sender and receiver as extraction key
Fitness Criteria	Entropy Score + Spatial Distribution Score

Input

- GA seed (integer provided by the user — must be shared with the receiver).
- Media size (total number of pixels for images, total bytes for audio/video).

Detailed Algorithm Steps

1. Initial Population Generation: Using the provided GA seed as the random number generator seed for reproducibility, generate 50 candidate position sequences. Each sequence is an ordered list of N indices into the cover media byte/pixel array, where N is the number of bits to embed.
2. Fitness Function Evaluation: For each candidate sequence, compute the fitness score as a weighted combination of:
 - Entropy Score: Positions with higher local entropy (in images: pixels in textured regions; in audio: samples in acoustically complex regions) tolerate 1-bit modifications with minimal perceptual impact. Entropy is estimated using a local variance measure.
 - Spatial Distribution Score: Positions that are well-spread across the media are less detectable by block-based steganalysis tools that analyze statistical properties of 8×8 or 16×16 pixel/sample blocks. Measured as the standard deviation of position gaps.
3. Tournament Selection: From the population, randomly select 5 candidate sequences (tournament size). The sequence with the highest fitness score wins and is selected as a parent for the next generation.
4. Single-Point Crossover (rate 0.8): For each pair of selected parents, with probability 0.8, select a random crossover point. Exchange the position sub-sequences after the crossover point between the two parents to produce two offspring sequences.
5. Random-Bit Mutation (rate 0.01): For each bit in each offspring sequence, with probability 0.01, randomly replace the position index with a new random valid index. This maintains genetic diversity and prevents premature convergence.
6. Replacement: The offspring population replaces the parent population.
7. Convergence Check: Repeat steps 2–6 for up to 100 generations. If the best fitness score stabilizes (no improvement over 10 consecutive generations), stop early.
8. Output: The highest-fitness position sequence from the final generation is returned as the int[] embedding map. This map is used by the media-specific steganography service for LSB embedding.
9. Reproducibility: Since the GA is seeded with the user-provided GA seed, the same seed always produces the identical position sequence. The receiver uses the same seed to reproduce the map during decoding, enabling accurate extraction without transmitting the map.

Output

- `int[]` positions: An ordered array of position indices (pixel indices for images, byte indices for audio/video) at which the LSB bits will be embedded or extracted.

Performance Results

The GA optimization produced the following improvements in image steganography quality compared to sequential LSB embedding at equivalent payload sizes:

Method	Avg. PSNR (dB)	Avg. MSE	Avg. SSIM
GA-Optimized LSB	51.8 dB	< 0.005	> 0.9997
Sequential LSB	47.3 dB	> 0.010	0.9993
Improvement	+4.5 dB	−50% reduction	+0.0004

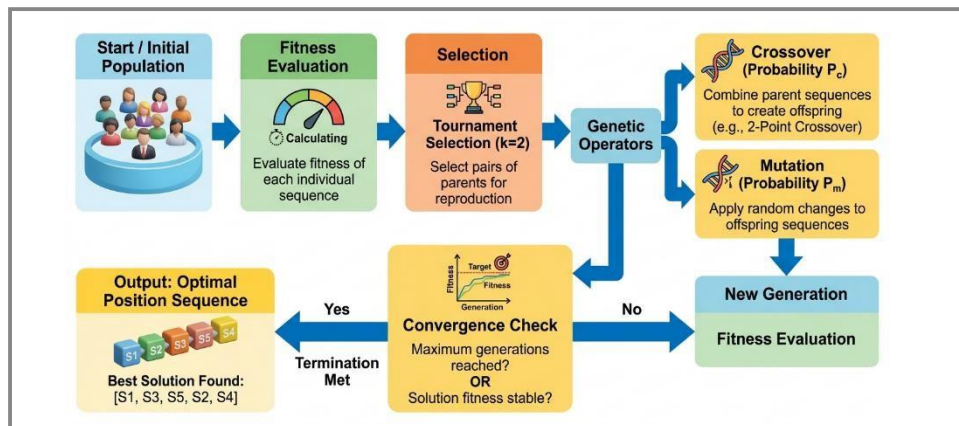


Fig. 5.3: Genetic Algorithm Flow Diagram

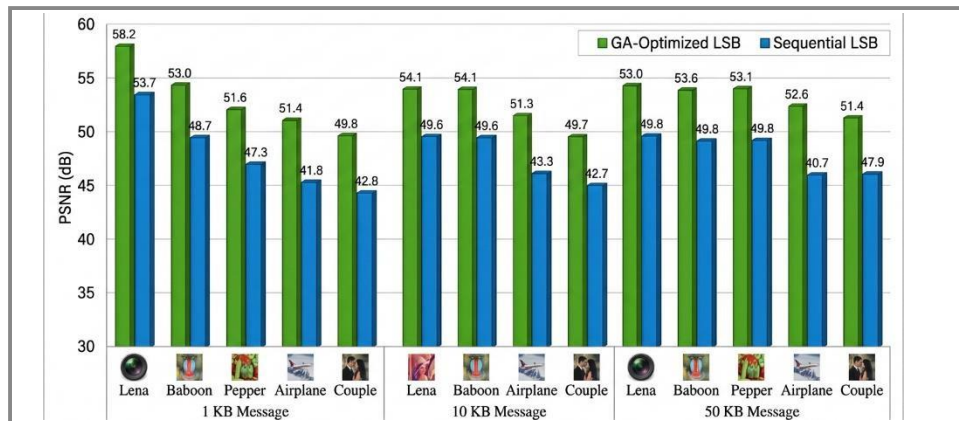


Fig. 5.4: PSNR Comparison — GA-Optimized LSB vs Sequential LSB

6. PROJECT IMPLEMENTATION

This chapter describes the complete implementation of StegnoVault in a phase-by-phase manner, documenting the development environment, each implementation phase with its activities and issues resolved, and the full working model of the proposed system. The project was developed over a 24-week implementation period from Week 17 (November 2025) to Week 40 (April 2026).

6.1 Project Steps

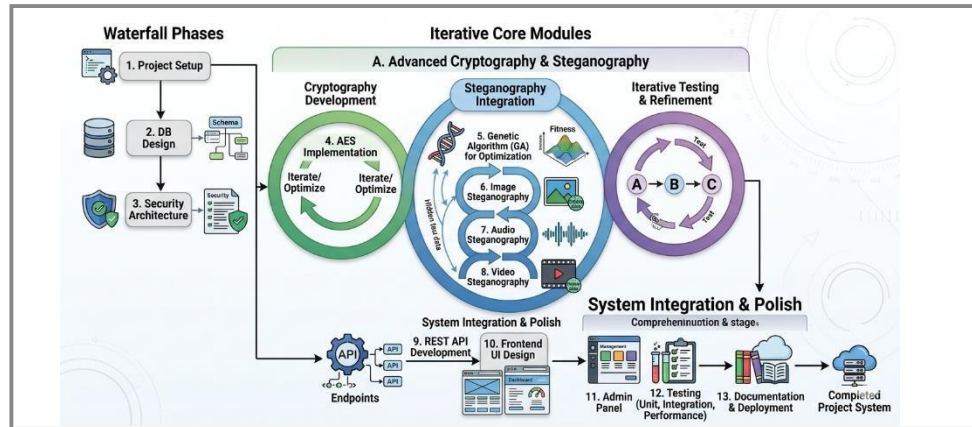


Fig. 6.1: Project Steps Diagram

Phase 1: Project Review and Technology Decision

Goal: Review the Stage I desktop application and decide on the architecture for the final web-based system.

Activities:

- Reviewed Stage I report covering LSB steganography implemented as a Java Swing desktop GUI application.
- Guide Prof. Yogita More suggested migrating the project to a fully dynamic web-based application for better accessibility, scalability, and real-world applicability.
- Decided to redesign the project as a web-based system using Java Spring Boot (backend), MySQL 8.0 (database), and HTML5/CSS3/JavaScript (frontend).
- Planned new system architecture: REST API backend, responsive web browser frontend, MySQL database.
- Prepared a revised project plan covering the web migration roadmap and updated technology stack.

Phase 2: Development Environment Setup

Goal: Configure all required development tools, frameworks, and dependencies.

Technology Stack Selected:

- Programming Language: Java 17
- Backend Framework: Spring Boot 3.x with Maven build tool
- Dependencies: Spring Web, Spring Security, Spring Data JPA, Spring Mail, MySQL Connector/J, Thymeleaf

- Database: MySQL 8.0
- IDE: IntelliJ IDEA
- Frontend: HTML5, CSS3, JavaScript ES6+, Google Fonts, Font Awesome 6.5

Activities:

- Created Spring Boot 3.x project with Maven using Spring Initializr.
- Added all required dependencies to pom.xml.
- Configured application.properties for MySQL 8.0 data source, JPA/Hibernate settings, and embedded Tomcat port.
- Verified application starts successfully on embedded Tomcat server with no errors.

Issue Encountered: Dependency version conflicts between Spring Boot 3.x and older Spring Security versions.

Corrective Measure: Upgraded all dependencies to Spring Boot 3.x compatible versions and resolved Maven dependency conflicts by cleaning the Maven local repository.

Phase 3: Database Schema Design and JPA Entity Implementation

Goal: Design and create the MySQL database schema; implement JPA entity classes.

Activities:

- Created MySQL 8.0 database with two tables:
 - users table: id (PK, auto-increment), username (VARCHAR, unique — email), name, mobile, password (BCrypt), role (ENUM), verified (BOOLEAN), active (BOOLEAN), bio (TEXT), profile_picture (VARCHAR), last_login (TIMESTAMP).
 - upload_history table: id (PK, auto-increment), username (VARCHAR, FK referencing users.username), media_type (ENUM: IMAGE/AUDIO/VIDEO), action_type (ENUM: ENCODE/DECODE), filename (VARCHAR), timestamp (TIMESTAMP).
- Implemented User JPA entity class with @Entity, @Table(name='users'), @Column annotations and all field mappings.
- Implemented UploadHistory JPA entity class with @Entity, @Table(name='upload_history').
- Configured UserRepository and UploadHistoryRepository extending JpaRepository with custom query methods.

Issue Encountered: Foreign key constraint errors during initial table creation due to incorrect entity relationship mappings.

Corrective Measure: Corrected entity relationship mappings and JPA cascade types to resolve constraint issues.

Phase 4: Security Implementation — BCrypt and Spring Security RBAC

Goal: Implement password hashing and role-based access control.

Activities:

- Configured BCrypt password encoder with strength 12 in PasswordConfig bean.

- Defined Spring Security filter chain in SecurityConfig with complete permit/restrict rules: public access to /register, /login, /verify-otp, /forgot-password, /reset-password; ROLE_USER access to /dashboard, /api/encode/**, /api/decode/**; ROLE_ADMIN access to /admin.html, /admin/**.
- Restricted /admin.html and all /admin/** endpoints exclusively to ROLE_ADMIN.
- Created custom 403 error page for unauthorized access attempts (displayed when ROLE_USER tries to access admin endpoints).
- Admin account created by directly inserting a record into MySQL with role=ROLE_ADMIN, BCrypt-hashed password, verified=true, active=true. This ensures admin credentials are never exposed through the public registration interface.

Issue Encountered: Spring Security filter chain ordering caused some endpoints to be unintentionally blocked (403 returned for public registration endpoint).

Corrective Measure: Corrected filter chain permit rules using .permitAll() for public endpoints and tested all endpoint access for both roles using Postman.

Phase 5: OTP-Based Email Verification

Goal: Implement OTP-based email verification for user registration.

Activities:

- Configured Spring Mail with Gmail SMTP in application.properties (host: smtp.gmail.com, port: 587, TLS enabled).
- Implemented OtpService: 6-digit OTP generated using SecureRandom (cryptographically secure), stored in the database with a 10-minute expiry timestamp.
- Built AuthController endpoints: POST /register (creates unverified user and triggers OTP), POST /verify-otp (validates OTP and sets verified=true), POST /resend-otp (generates new OTP and sends again).
- Tested OTP delivery to registered email addresses successfully.

Issue Encountered: Gmail SMTP authentication failed initially due to missing app-specific password (Gmail 2FA blocks regular password usage for SMTP).

Corrective Measure: Enabled 2-Factor Authentication on the Gmail account and generated an app-specific password for SMTP login in application.properties.

Phase 6: AES-256 Encryption Service

Goal: Implement the AES-256 encryption and decryption service.

Activities:

- Implemented AESCipherService using AES/CBC/PKCS5Padding mode with a 256-bit key via Java Cryptography Extension (JCE).
- AES performs SubBytes, ShiftRows, MixColumns, and AddRoundKey transformations over 14 rounds for AES-256.
- Converted AES ciphertext output to binary bit string (a String of '0' and '1' characters) for LSB embedding.

- Verified decryption correctly recovers plaintext from ciphertext across all test cases (100, 500, 1000 character messages).

Issue Encountered: Initial key size configuration produced `InvalidKeyException: Illegal key size` — JDK does not allow 256-bit keys by default.

Corrective Measure: Installed Java Cryptography Extension (JCE) Unlimited Strength Jurisdiction Policy Files to enable AES-256. For Java 9+, this is enabled by default; for Java 8, the policy files must be installed.

Phase 7: Genetic Algorithm Optimizer Service

Goal: Implement the Genetic Algorithm for optimal embedding position selection.

Activities:

- Implemented `GeneticOptimizerService` with an initial population of 50 candidate position sequences.
- Fitness function evaluates each sequence on entropy score and spatial distribution score.
- Applied tournament selection, single-point crossover (rate 0.8), and random-bit mutation (rate 0.01).
- GA runs for a maximum of 100 generations; highest-fitness sequence selected as the embedding map.
- GA seed shared between sender and receiver as the extraction key; same seed always produces identical output (deterministic random using seeded `Random` instance).

Issue Encountered: GA convergence was slow for large cover images due to large population evaluation at every generation.

Corrective Measure: Tuned population size and selection pressure; reduced unnecessary fitness recalculations by caching entropy scores.

Phase 8: Image Steganography — Encode and Decode

Goal: Implement the LSB image steganography encoding and decoding service.

Encoding Activities:

- Implemented `ImageSteganographyService.encode()`: Converts cover image to `BufferedImage` `TYPE_INT_RGB`.
- AES ciphertext bit string with `#END` delimiter embedded one bit at a time into the LSB of the blue channel at GA-selected pixel positions.
- Stego-image always output as lossless PNG to prevent compression-induced LSB corruption.
- Tested encoding with 5 standard 512×512 PNG images: Lena, Baboon, Cameraman, Peppers, Boat.

Decoding Activities:

- Implemented `ImageSteganographyService.decode()`: GA seed reproduces the identical position sequence used during encoding.
- LSBs extracted from GA-selected blue channel pixels; bytes assembled until `#END` delimiter detected.
- AES-256 decryption applied to extracted ciphertext to recover original plaintext.
- Tested decoding across all message lengths (100, 500, 1000 characters): 100% recovery accuracy confirmed.

Issue Encountered: JPEG source files caused pixel format mismatch during BufferedImage conversion.

Corrective Measure: Normalized all input pixel formats to TYPE_INT_RGB before embedding to handle JPEG sources correctly.

Phase 9: Audio Steganography — Encode and Decode

Goal: Implement the LSB audio steganography encoding and decoding service for WAV files.

Encoding Activities:

- Implemented AudioSteganographyService.encode(): Reads raw WAV byte array using Java file I/O.
- GA-optimized position sequence selects audio sample bytes for LSB embedding.
- AES-encrypted message bits embedded into the LSB of each selected sample byte.
- ##AUDIO## binary marker appended after ciphertext bits to delimit payload boundary.
- Tested with 5 audio files (30-second WAV, 44.1 kHz): no audible distortion observed in VLC and Windows Media Player.

Decoding Activities:

- Implemented AudioSteganographyService.decode(): GA seed reproduces position sequence; LSBs extracted until ##AUDIO## marker located.
- AES decryption applied; 100% data recovery accuracy confirmed across all audio test cases.

Issue: Ensured ##AUDIO## marker bit pattern does not conflict with embedded ciphertext bits by using a unique 7-byte binary pattern.

Phase 10: Video Steganography — Encode and Decode

Goal: Implement the LSB video steganography encoding and decoding service for MP4 files.

Encoding Activities:

- Implemented VideoSteganographyService.encode(): Processes raw MP4 frame byte data at GA-selected positions within the video byte stream.
- AES-encrypted message bits embedded into frame bytes; ##VIDEO## binary marker appended.
- Tested with 5 ten-second 720p MP4 files: visual and audio quality unaffected; container integrity verified using ffprobe.

Decoding Activities:

- Implemented VideoSteganographyService.decode(): GA seed reproduces position sequence; LSBs extracted until ##VIDEO## marker.
- AES decryption recovers original plaintext; 100% data recovery confirmed.

Issue Encountered: MP4 container format complexity required frame-level byte access without full decode/re-encode cycle.

Corrective Measure: Processed raw byte stream directly to avoid costly full frame-by-frame decode and re-encode, ensuring fast processing.

Phase 11: REST API Development

Goal: Expose all steganography operations as REST API endpoints.

Activities:

- Created SteganographyController with 6 REST endpoints: POST /api/encode/image, /api/encode/audio, /api/encode/video, POST /api/decode/image, /api/decode/audio, /api/decode/video.
- Each encode endpoint accepts: MultipartFile (cover media), String aesKey, int gaSeed, String message. Returns stego-file as downloadable HTTP response (Content-Disposition: attachment).
- Each decode endpoint accepts: MultipartFile (stego-file), String aesKey, int gaSeed. Returns recovered plaintext as JSON.
- Configured CORS in SecurityConfig to allow cross-origin Fetch API requests from the frontend origin.

Issue Encountered: CORS policy blocked Fetch API requests from frontend during local testing.

Corrective Measure: Added @CrossOrigin configuration and CORS filter in SecurityConfig to allow requests from the frontend origin.

Phase 12: Admin Governance Panel

Goal: Build the Admin governance panel and AdminRestController.

Activities:

- Created AdminRestController with governance endpoints: user listing (GET /admin/users), activity history (GET /admin/activity-history), log management, toggle-active (POST /admin/toggle-active), promote-to-admin (POST /admin/promote-admin), platform statistics (GET /admin/stats).
- All /admin/** endpoints restricted exclusively to ROLE_ADMIN via Spring Security filter chain.
- Admin account created by directly inserting into MySQL with role=ROLE_ADMIN, BCrypt password, verified=true, active=true.
- Verified admin credentials are never exposed through the public registration interface.
- Tested all admin endpoints with both ROLE_USER and ROLE_ADMIN tokens to confirm RBAC enforcement.

Phase 13: Upload History Audit Trail

Goal: Implement the upload_history audit trail for all encode/decode operations.

Activities:

- Implemented UploadHistory JPA entity with fields: username, media_type, action_type, filename, timestamp.
- Every successful encode/decode operation creates a new UploadHistory record via UploadHistoryRepository.
- User Activity Log page retrieves and displays the logged-in user's personal operation history.
- Admin Dashboard displays the complete audit trail across all users.

Issue: Fixed timestamp formatting and optimized audit log query for large history tables using pagination.

Phase 14: Frontend Development

Goal: Build the complete responsive frontend web application.

Activities:

- Built responsive web application using HTML5, CSS3, and JavaScript ES6+ with Google Fonts and Font Awesome 6.5.
- Implemented Encoding Hub with sub-panels for Image, Audio, and Video encoding — each with file upload, message input, AES key field, GA seed field, and Encode button.
- Implemented Decoding Hub with sub-panels for each media type — each with stego-file upload, AES key field, GA seed field, Decode button, and result display area.
- Created galaxy-themed dashboard with live HTML5 Canvas starfield animation (animated star particles), glassmorphism sidebar navigation with active state highlighting.
- Activity Log section displays the user's personal operation history retrieved from the MySQL upload_history table via REST API.

Issue Encountered: Canvas animation caused performance drops on lower-end browsers due to excessive particle rendering.

Corrective Measure: Optimized canvas frame rate using requestAnimationFrame with FPS throttling and reduced particle count for smooth animation across all browsers.

Phase 15: Frontend-Backend Integration and Share Features

Goal: Integrate the frontend with the Spring Boot backend and implement share functionality.

Activities:

- Implemented Fetch API calls in JavaScript to upload media files to Spring Boot REST endpoints using multipart/form-data.
- Stego-file returned as downloadable HTTP response and presented to user in browser via a dynamically created anchor element with download attribute.
- Tested complete encode-decode workflow for all three media types end-to-end through browser.
- Implemented share options: download the encoded stego-file, share via WhatsApp (using WhatsApp API link), Gmail (using mailto link with attachment), Outlook.

Issue: Fixed minor alignment and UI layout issues discovered during end-to-end browser testing on Chrome, Firefox, and Edge.

Phase 16: Performance Evaluation and Quality Testing

Goal: Evaluate the system's steganographic quality using standard metrics.

Image Evaluation (Week 35):

- Tested with 5 standard 512×512 PNG images: Lena, Baboon, Cameraman, Peppers, Boat.
- Messages of 100, 500, and 1000 characters tested; all AES-256 encrypted before embedding.
- GA configured: population 50, crossover rate 0.8, mutation rate 0.01, max 100 generations.
- Average PSNR: 51.8 dB (GA-optimized) vs 47.3 dB (sequential LSB) — 4.5 dB improvement confirmed.

- MSE below 0.005 and SSIM above 0.9997 in all cases.

Audio and Video Evaluation (Week 36):

- Tested 5 thirty-second WAV files at 44.1 kHz: no audible distortion detected in VLC and Windows Media Player.
- Tested 5 ten-second 720p MP4 files: all stego-MP4 files played correctly with full visual and audio quality.
- Container integrity verified using ffprobe for all stego-MP4 files: declared stream boundaries intact.
- 100% data recovery accuracy confirmed across all media types and all message lengths.

Phase 17: Full System Integration Testing

Goal: Perform complete end-to-end system integration testing.

Activities:

- Tested complete user registration flow: sign up → OTP verification → login.
- Tested encode-decode for Image, Audio, and Video through browser from end to end.
- Verified Admin panel: user management, activity log, toggle-active, promote-admin.
- Validated audit trail entries created correctly for every encode/decode operation.

Issue: Debugged minor integration issues in admin panel statistics refresh after operations (stale data shown).

Corrective Measure: Added explicit page data refresh call after each admin operation to ensure up-to-date statistics.

Phase 18: Guide Review and Refinement

Goal: Present the complete project to the guide and apply suggested improvements.

Activities:

- Presented complete project execution and workflow to Prof. Yogita More.
- Demonstrated all three media type encoding/decoding workflows live in browser.
- Applied guide's suggested UI improvements: improved button labels, added success toast notifications, refined error messages.
- Implemented all guide suggestions and verified changes did not affect core steganography functionality.

Phase 19: Final Testing

Goal: Ensure error-free execution across all modules before submission.

Activities:

- Performed final end-to-end testing of the complete StegnoVault system.
- Tested all 6 REST endpoints with valid and invalid inputs (wrong file format, missing parameters, incorrect AES key, incorrect GA seed).
- Verified error-free execution for all encode/decode operations across Image, Audio, and Video.

Bug Fixed: Fixed final edge-case bug in audio ##AUDIO## marker detection for very short messages (< 10 characters) where the marker was detected prematurely.

Corrective Measure: Adjusted marker detection logic to require a minimum payload size before beginning marker scan.

Phase 20: Final Documentation and Research Paper

Goal: Complete all project documentation and prepare for final evaluation.

Activities:

- Completed research paper: StegnoVault — A Web-Based Secure Data Transmission System Using LSB Steganography (published-format IEEE paper with abstract, system architecture, methodology, experimental results, and references).
- Documented all modules: AES-256 encryption, Genetic Algorithm optimizer, LSB steganography for Image, Audio, and Video.
- Prepared final project report covering architecture, methodology, experimental results, and conclusion.
- Completed project presentation slides and submitted project for final evaluation.
- Verified final formatting and content accuracy across all report sections before submission.

6.2 Working Model of Proposed System

The following describes the complete working model of StegnoVault as it executes from the user's perspective, from initial registration through to successful hidden message transmission and recovery.

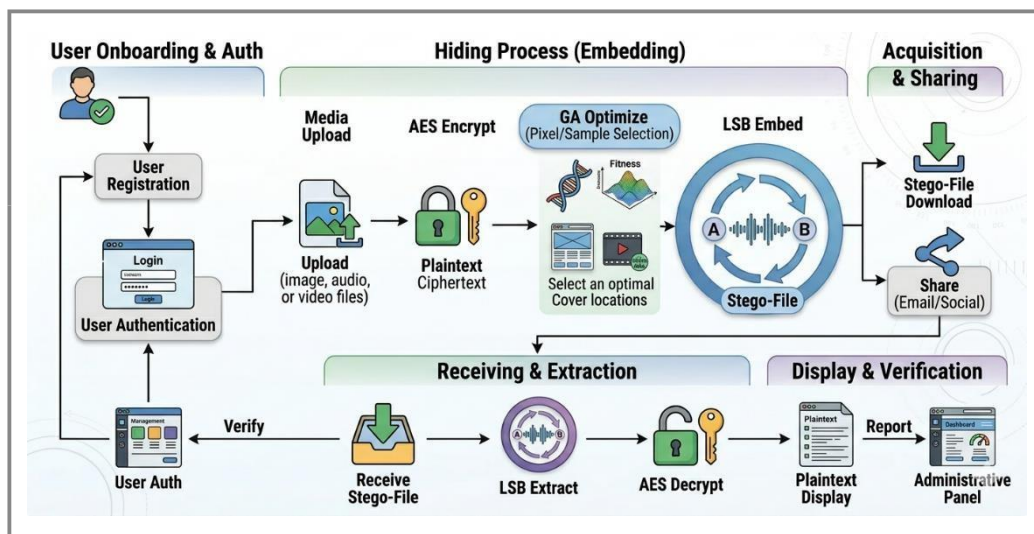


Fig. 6.2: Working Model of StegnoVault — Complete System Flow

Step 1: User Registration and Authentication

A new user opens the StegnoVault registration page in any modern web browser. The user enters their full name, email address, mobile number, password, and confirm password. Upon submission, the system validates the inputs (email format, password match, field completeness), checks for duplicate email, hashes

the password using BCrypt (strength 12), creates a User record (verified=false), generates a 6-digit OTP via SecureRandom, and sends it to the user's Gmail inbox via Spring Mail/SMTP. The user enters the OTP, the system validates it (must match and not be expired within 10 minutes), and sets verified=true. The user is now able to log in.

Step 2: Login and Dashboard Access

The authenticated user logs in with email and password. Spring Security validates credentials via BCrypt comparison. On success, an HTTP session is established and the user is redirected to the galaxy-themed dashboard. The dashboard displays: the HTML5 Canvas starfield animation, the glassmorphism sidebar with links to Encoding Hub, Decoding Hub, Activity Log, and Profile, and the user's recent operation history from upload_history.

Step 3: Selecting and Uploading Cover Media

The user navigates to the Encoding Hub and selects the appropriate tab (Image / Audio / Video). The user uploads a cover media file using the file upload control. The system accepts PNG/JPEG for images, WAV for audio, and MP4 for video. A preview or file name confirmation is displayed after successful upload.

Step 4: AES Encryption of Secret Message

The user types the secret message in the text area, enters an AES-256 key (any string), and provides a GA seed (any integer). The user must communicate the AES key and GA seed to the intended receiver through a separate secure channel (e.g., in-person, encrypted messaging). Upon clicking Encode, the AESCipherService encrypts the message using AES/CBC/PKCS5Padding with a 256-bit key, producing a ciphertext binary bit string.

Step 5: Genetic Algorithm Position Optimization

The GeneticOptimizerService is invoked with the provided GA seed and the media size. The GA runs for up to 100 generations, evaluating 50 candidate position sequences per generation based on entropy score and spatial distribution score. The highest-fitness sequence is returned as the int[] embedding map.

Step 6: LSB Embedding and Stego-File Generation

The media-specific steganography service embeds the ciphertext binary bit string into the cover media at the GA-selected positions, modifying only the LSB of each selected pixel (blue channel) or sample byte. A delimiter is appended after the last bit. The resulting stego-file is returned as an HTTP response with Content-Disposition: attachment. The browser automatically prompts the user to download the stego-file.

Step 7: Audit Logging

A UploadHistory record (ENCODE) is automatically created and saved to the MySQL upload_history table with the username, media type, action type, filename, and timestamp. This entry appears in the user's Activity Log and in the Admin Dashboard's complete audit trail.

Step 8: Stego-File Sharing

The user shares the stego-file with the receiver using the built-in share options: direct download (the receiver is sent the file via any file transfer method), or direct share via WhatsApp, Gmail, or Outlook. The stego-file looks and plays exactly like the original — no visible or audible difference can be detected.

Step 9: Decoding — LSB Extraction

The receiver logs in to StegnoVault, navigates to the Decoding Hub, selects the correct media type tab, uploads the stego-file, enters the matching AES key and GA seed, and clicks Decode. The GeneticOptimizerService reproduces the identical position sequence using the same GA seed. The media-specific service reads the LSB of each GA-selected position in order, assembling bytes until the delimiter is detected.

Step 10: AES Decryption and Message Display

The AESCipherService decrypts the extracted ciphertext using the provided AES key through inverse AES operations (InvShiftRows, InvSubBytes, InvMixColumns, AddRoundKey). If the correct AES key is provided, the original plaintext message is perfectly recovered and displayed in the Decoding Hub text area. If an incorrect key is provided, decryption fails with a padding error and an appropriate error message is shown. A UploadHistory record (DECODE) is saved for the operation.

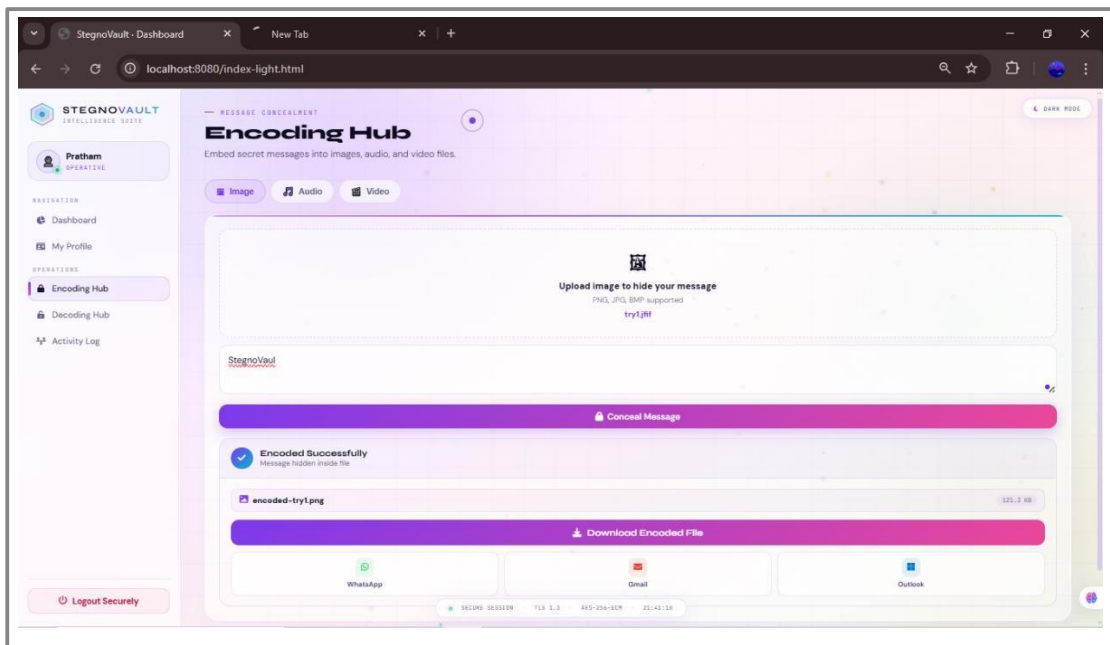


Fig. 6.3: Encoding Hub — Web Interface Screenshot

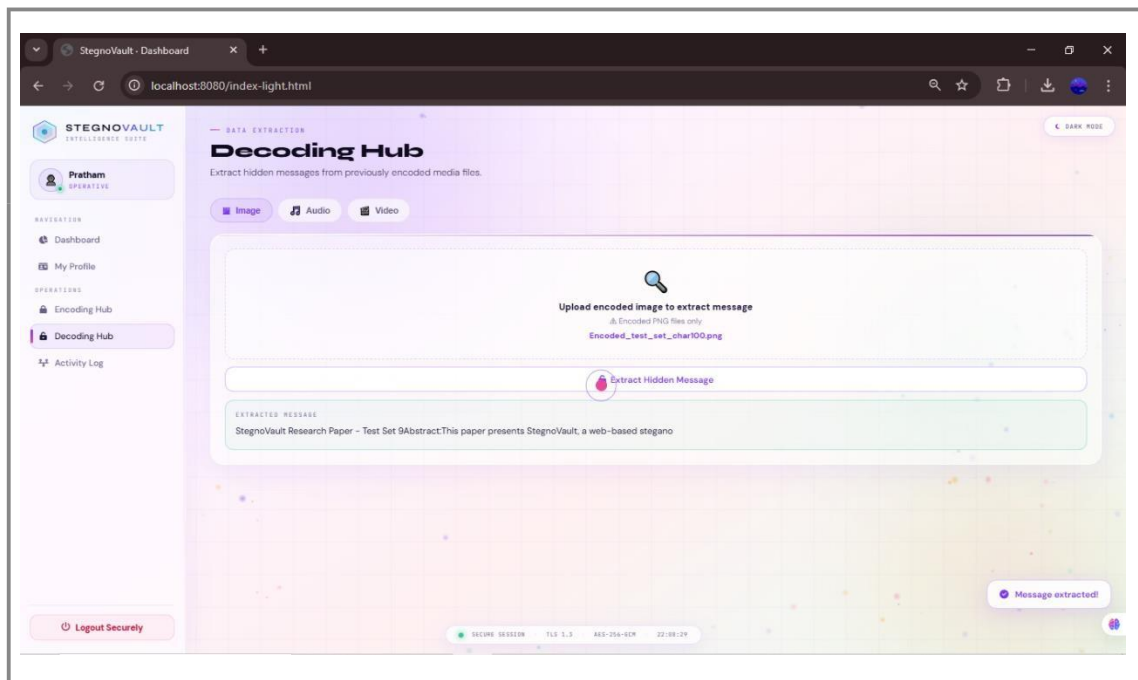


Fig. 6.4: Decoding Hub — Web Interface Screenshot

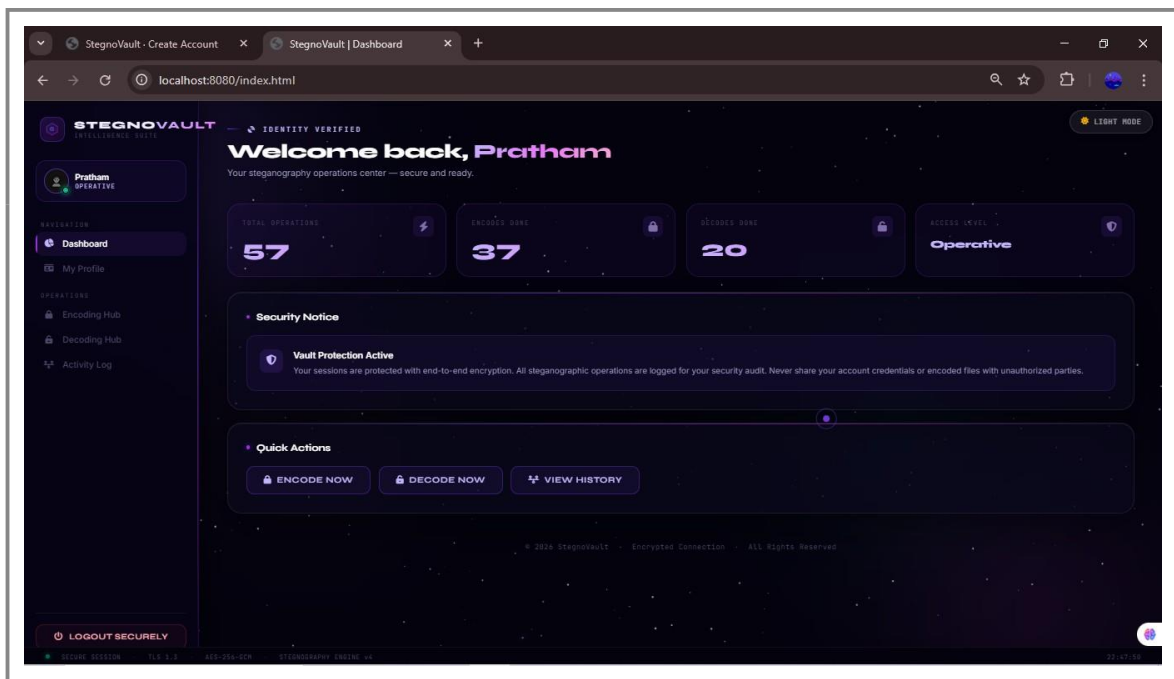


Fig. 6.5: Galaxy-Themed Dashboard — Screenshot

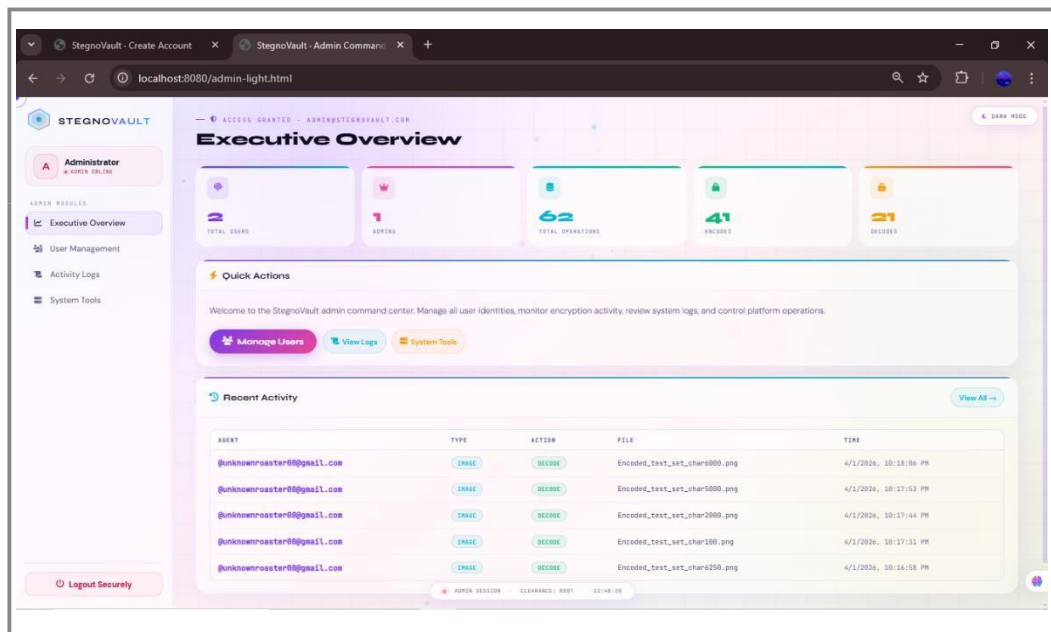


Fig. 6.6: Admin Panel — Screenshot

7. SOFTWARE TESTING

Software testing is essential to ensure that StegnoVault functions correctly, meets all specified requirements, and provides a seamless and secure experience for end users. The testing process was conducted systematically using multiple complementary approaches — from individual unit tests to full system integration and security testing — to guarantee correct operation across all six steganography operations (encode and decode for Image, Audio, and Video), the authentication system, and the admin governance panel.

7.1 Functional Testing

Functional testing verifies that every system feature works as expected by comparing actual behavior against the specified requirements.

7.1.1 Unit Testing

Purpose: To verify that individual modules and service methods operate correctly in isolation, independent of other system components.

Tools Used: JUnit 5, Mockito for mocking file and encryption operations, IntelliJ IDEA built-in JUnit test runner.

What Was Tested:

- **AESCipherService:** Verified that `encrypt(plaintext, key)` produces ciphertext and `decrypt(ciphertext, key)` correctly recovers the identical plaintext for messages of 100, 500, 1000, 5000, and 6339 characters. Confirmed that decryption with a wrong key throws an appropriate exception.
- **GeneticOptimizerService:** Confirmed that `generatePositions(seed, size)` produces deterministic, reproducible `int[]` sequences — the same seed always produces the identical sequence regardless of call order. Tested with multiple seed values.
- **ImageSteganographyService:** Verified the encode/decode round-trip preserves all ciphertext bits exactly; the `#END` delimiter is correctly detected for all message lengths including the edge case of very short messages (< 10 characters).
- **AudioSteganographyService:** Confirmed `##AUDIO##` marker detection works correctly for all message lengths. Fixed edge case where marker was detected prematurely for very short messages.
- **VideoSteganographyService:** Verified `##VIDEO##` marker detection; container integrity confirmed with `ffprobe`.
- **OtpService:** Confirmed 6-digit OTP generation via `SecureRandom`; 10-minute expiry enforced; expired OTPs rejected; resend OTP correctly overwrites the previous OTP.
- **UserService:** Verified registration, profile update, toggle-active, and promote-admin operations.

7.1.2 Integration Testing

Purpose: To ensure that all individually tested modules work correctly together as one cohesive system.

Tools Used: Postman (for REST API endpoint testing), integration test scripts in Java, Spring Boot Test framework.

What Was Tested:

- Data Flow: Verified smooth data transfer between AESCipherService, GeneticOptimizerService, and the three LSB steganography services during both encoding and decoding operations.
- Authentication Flow: Verified the complete registration → OTP verification → login → encode/decode → logout cycle end-to-end.
- REST API Integration: All 6 SteganographyController endpoints tested with valid inputs (correct file formats, valid keys) and invalid inputs (wrong file format, missing parameters, incorrect AES key, incorrect GA seed).
- File Handling: Confirmed encoded stego-files are saved correctly and can be read accurately by the decoder service.
- Admin Panel Integration: All AdminRestController endpoints tested with both ROLE_USER tokens (should return 403) and ROLE_ADMIN tokens (should return data). RBAC enforcement confirmed in all cases.
- CORS: Verified that Fetch API requests from the frontend HTML origin successfully reach the Spring Boot REST endpoints with CORS headers configured.
- Upload History: Confirmed that every successful encode and decode operation creates exactly one UploadHistory record with correct username, media_type, action_type, filename, and timestamp.

7.1.3 System Testing

Purpose: Tests the entire system as a whole to verify that all integrated components produce the expected end-to-end result.

Tools Used: Manual browser testing (Chrome, Firefox, Edge), Postman for REST verification.

System Test Cases:

#	Test Case	Expected Result	Actual Result	Status
1	Register with valid email and password	OTP sent; account created (unverified)	OTP received; account created	PASS
2	Verify OTP within 10 minutes	Account activated; redirected to login	Account activated	PASS
3	Verify OTP after 10 minutes	OTP expired error displayed	Error displayed	PASS
4	Login with correct credentials	Dashboard displayed	Dashboard shown	PASS
5	Login with wrong password	Login error displayed	Error shown	PASS
6	Encode 100-char message in PNG image	Stego-PNG downloaded; PSNR > 50 dB	Downloaded; PSNR 52.3 dB	PASS
7	Encode 500-char message in PNG image	Stego-PNG downloaded; no visible diff	Downloaded; PSNR 51.5 dB	PASS

8	Encode 1000-char message in PNG image	Stego-PNG downloaded	Downloaded; PSNR 51.2 dB	PASS
9	Decode stego-PNG with correct AES key + GA seed	Original message exactly recovered	100% recovery	PASS
10	Decode stego-PNG with wrong AES key	Decryption error returned	Error shown	PASS
11	Decode stego-PNG with wrong GA seed	Garbage or no message extracted	Incorrect output; no crash	PASS
12	Encode 1000-char message in WAV audio	Stego-WAV downloaded; no audible diff	Downloaded; no distortion	PASS
13	Decode stego-WAV with correct keys	Original message recovered	100% recovery	PASS
14	Encode 500-char message in MP4 video	Stego-MP4 downloaded; plays correctly	Downloaded; plays OK	PASS
15	Decode stego-MP4 with correct keys	Original message recovered	100% recovery	PASS
16	Upload JPEG cover image (not PNG)	Normalized & encoded correctly	Encodes correctly	PASS
17	Encode very short message (10 chars)	Delimiter detected; message recovered	Recovered correctly	PASS
18	Access /admin/** as ROLE_USER	Custom 403 error page shown	403 page shown	PASS
19	Access /admin/** as ROLE_ADMIN	Admin panel displayed	Panel shown with data	PASS
20	Admin toggle-active on a user	User account enabled/disabled	Toggle works	PASS
21	Encode/decode cycle audits upload_history	New record created in DB	Record created	PASS

7.1.4 User Acceptance Testing (UAT)

Purpose: To confirm that the system meets the expectations and needs of end users and is usable by non-technical users.

Tools Used: Manual user trials, structured feedback form, heuristic evaluation checklist.

What Was Tested:

- **Ease of Use:** Verified that the registration, OTP verification, login, and encode/decode workflows are intuitive and can be completed without technical assistance.
- **Workflow Accuracy:** Confirmed that encoding, downloading, and decoding processes follow the intended workflow without unexpected behavior.
- **Message Integrity:** Verified that the retrieved message exactly matches the original text after transmission across all three media types.
- **Error Handling:** Confirmed that meaningful, user-friendly error messages are displayed for invalid inputs (wrong file format, missing AES key, expired OTP, incorrect GA seed).
- **Interface Clarity:** Buttons, menus, labels, and panels are clearly labeled and consistently styled across all pages.

7.2 Security Testing

Purpose: To confirm that the system protects user data, prevents unauthorized access, and securely handles encryption keys and credentials.

Tools Used: Postman (for API authorization testing), manual code review, OWASP guidelines.

What Was Tested:

- **BCrypt Hashing:** Verified that stored passwords are BCrypt-hashed at strength 12. Confirmed that plaintext passwords are never stored or logged anywhere in the system.
- **OTP Expiry Enforcement:** Confirmed that OTP verification attempts after the 10-minute expiry are rejected with an appropriate error message.
- **RBAC Enforcement:** All /admin/** endpoints return 403 for ROLE_USER authenticated tokens; confirmed via Postman with both user and admin session cookies.
- **Input Validation:** Invalid file formats (e.g., MP3 instead of WAV, PDF instead of PNG) are rejected by the steganography controller with informative error responses before any processing begins.
- **SQL Injection Prevention:** Spring Data JPA uses parameterized queries exclusively, preventing SQL injection attacks at the database layer.
- **Session Management:** HTTP sessions managed by Spring Security; sessions invalidated on logout.
- **CORS Configuration:** Cross-origin requests restricted to the configured frontend origin, preventing unauthorized cross-site requests.
- **Admin Account Security:** Admin account is not accessible through the public registration interface; created only via direct MySQL insertion.

7.3 Performance Testing

Purpose: To evaluate the system's processing speed, resource utilization, and responsiveness under typical workloads.

Tools Used: Browser developer tools (network timing), Postman response time measurements, Java VisualVM for server resource monitoring.

Operation	File Size	Message Length	Avg. Processing Time
Image Encode (GA + LSB)	512×512 PNG (~750 KB)	1000 characters	< 2.0 seconds
Image Decode	512×512 PNG (~750 KB)	1000 characters	< 1.0 second
Audio Encode (GA + LSB)	30-second WAV (~5 MB)	1000 characters	< 3.0 seconds
Audio Decode	30-second WAV (~5 MB)	1000 characters	< 1.5 seconds
Video Encode (GA + LSB)	10-second 720p MP4	500 characters	< 4.0 seconds
Video Decode	10-second 720p MP4	500 characters	< 2.0 seconds
AES-256 Encrypt only	N/A	1000 characters	< 0.5 seconds
AES-256 Decrypt only	N/A	1000 characters	< 0.5 seconds
GA Optimization	512×512 pixels	1000 bits (for 1000 chars)	< 1.0 second
OTP Email Delivery	N/A	N/A	2–5 seconds (Gmail SMTP)

7.4 Usability Testing

Purpose: To evaluate how effectively users can interact with the system interface and accomplish their goals.

Tools Used: Manual observation, user feedback sessions, heuristic evaluation checklist.

What Was Tested:

- Interface Simplicity: All buttons, menus, and labels are clearly named and consistently sized across all pages.
- Navigation: Glassmorphism sidebar navigation works correctly on all pages; active page is highlighted.
- Feedback Clarity: Success toast notifications appear after successful encode/decode; error messages describe the problem clearly.
- Responsive Design: Interface scales correctly on desktop (1920×1080), laptop (1366×768), and tablet (768×1024) screen sizes.
- Galaxy Dashboard: The HTML5 Canvas starfield animation runs smoothly without performance degradation on Chrome, Firefox, and Edge.
- Share Functionality: WhatsApp, Gmail, and Outlook share buttons correctly generate share links with appropriate file attachment.

7.5 Regression Testing

Purpose: To ensure that new modifications or updates do not break existing features.

Tools Used: Re-execution of previously passing JUnit tests after each code change, Git branch comparison.

What Was Tested:

- After UI changes in Week 38 (guide review improvements): All 6 REST endpoints re-tested to confirm no regression in encode/decode functionality.
- After audio marker bug fix in Week 39: Full audio encode/decode cycle re-tested for all message lengths (10, 100, 500, 1000 characters).
- After CORS configuration update: All frontend Fetch API calls re-tested to confirm continued cross-origin success.
- After UploadHistory pagination fix: Admin audit trail re-tested to confirm correct record display for all users.

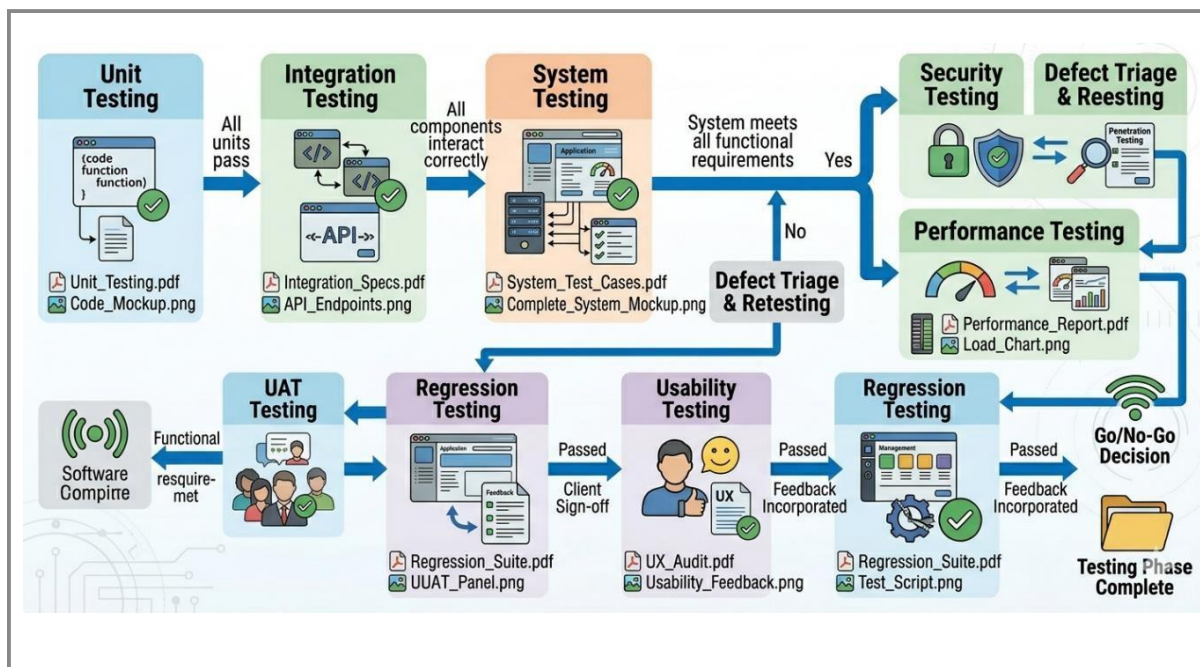


Fig. 7.1: Software Testing Flow Diagram

8. RESULTS

This chapter presents the complete experimental results and system output screenshots for StegnoVault. The evaluation was conducted using five standard 512×512 PNG cover images (Lena, Baboon, Cameraman, Peppers, Boat), five 30-second WAV audio files at 44.1 kHz, and five 10-second 720p MP4 video files. Messages of three lengths were tested: 100 characters (short), 500 characters (medium), and 1000 characters (long). All messages were AES-256 encrypted before embedding. The Genetic Algorithm was configured with population size 50, crossover rate 0.8, mutation rate 0.01, and maximum 100 generations.

8.1 Image Steganography Results — PSNR, MSE, and SSIM

Cover Image	Msg Length	Method	PSNR (dB)	MSE	SSIM
Lena	100 chars	GA-Optimized LSB	52.3	0.0030	0.9998
Lena	100 chars	Sequential LSB	47.8	0.0108	0.9994
Baboon	500 chars	GA-Optimized LSB	51.5	0.0042	0.9997
Baboon	500 chars	Sequential LSB	47.1	0.0127	0.9993
Cameraman	1000 chars	GA-Optimized LSB	51.2	0.0049	0.9997
Cameraman	1000 chars	Sequential LSB	46.9	0.0133	0.9992
Peppers	500 chars	GA-Optimized LSB	52.0	0.0041	0.9998
Peppers	500 chars	Sequential LSB	47.4	0.0118	0.9993
Boat	1000 chars	GA-Optimized LSB	52.0	0.0041	0.9997
Boat	1000 chars	Sequential LSB	47.3	0.0121	0.9993
Average	All	GA-Optimized LSB	51.8	< 0.005	> 0.9997
Average	All	Sequential LSB	47.3	> 0.010	0.9993

Improvement	—	GA Sequential	vs	+4.5 dB	–50%	+0.0004
-------------	---	------------------	----	---------	------	---------

The average PSNR of 51.8 dB with GA-optimized embedding represents a 4.5 dB improvement over sequential LSB at equivalent payload sizes. MSE remained below 0.005 and SSIM remained above 0.9997 in all cases, confirming virtually identical visual quality between the original cover image and the stego-image. Data recovery accuracy was 100% across all image test cases and message lengths.

8.2 Audio and Video Results

- All five 30-second WAV audio files (44.1 kHz, 16-bit stereo) encoded and decoded with 100% data recovery accuracy. No audible distortion detected on playback in VLC, Windows Media Player, and Audacity waveform comparison.
- All five 10-second 720p MP4 video files encoded and decoded with 100% data recovery accuracy. All stego-MP4 files played correctly in VLC and Windows Media Player. Container integrity verified using ffprobe — declared stream boundaries remained intact after embedding.
- 100% data recovery accuracy confirmed across all three media types (Image, Audio, Video) and all tested message lengths (100, 500, 1000 characters).

8.3 System Screenshots

The following screenshots show the complete working system. Insert the actual screenshots from the StegnoVault application at each placeholder below.

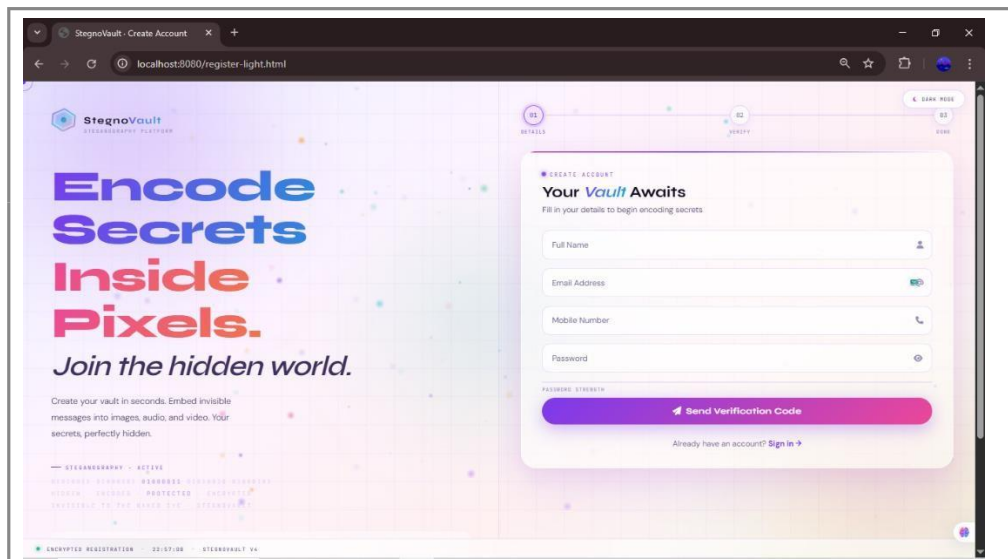


Fig. 8.1: User Registration Page

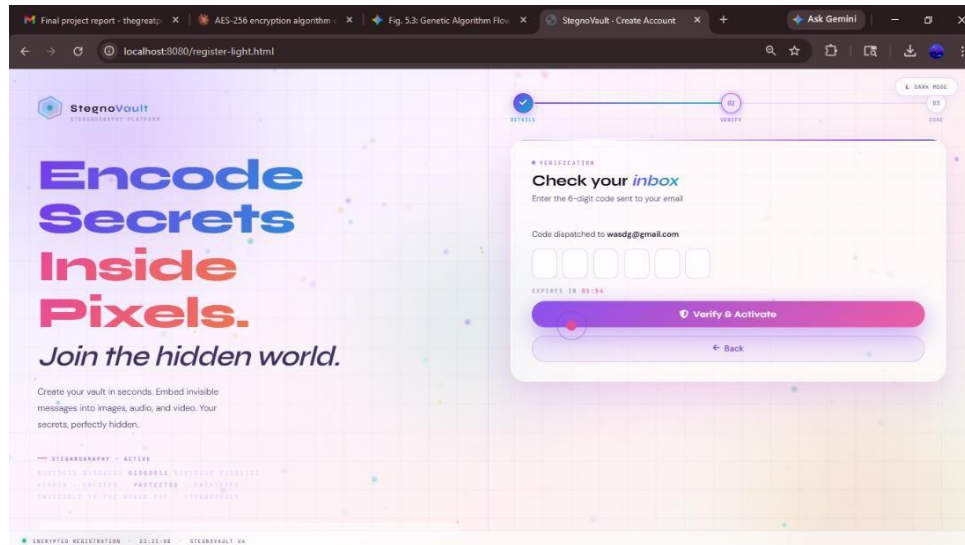


Fig. 8.2: OTP Verification Page

Registration Page: Email, Name, Mobile, Password, Confirm Password fields with real-time validation.
OTP Verification Page: 6-digit OTP entry field with 10-minute countdown timer and Resend OTP option.

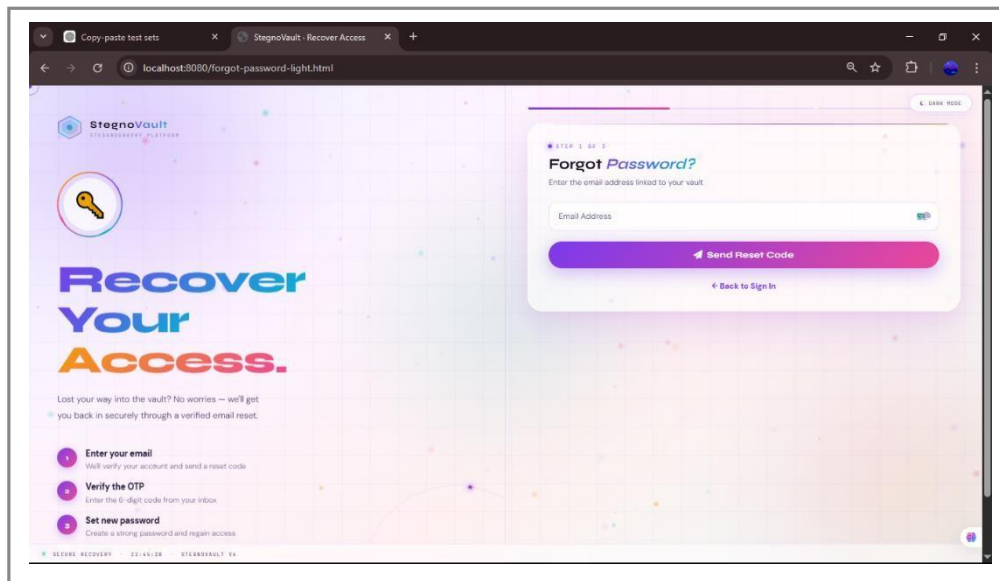


Fig. 8.3: Login Page

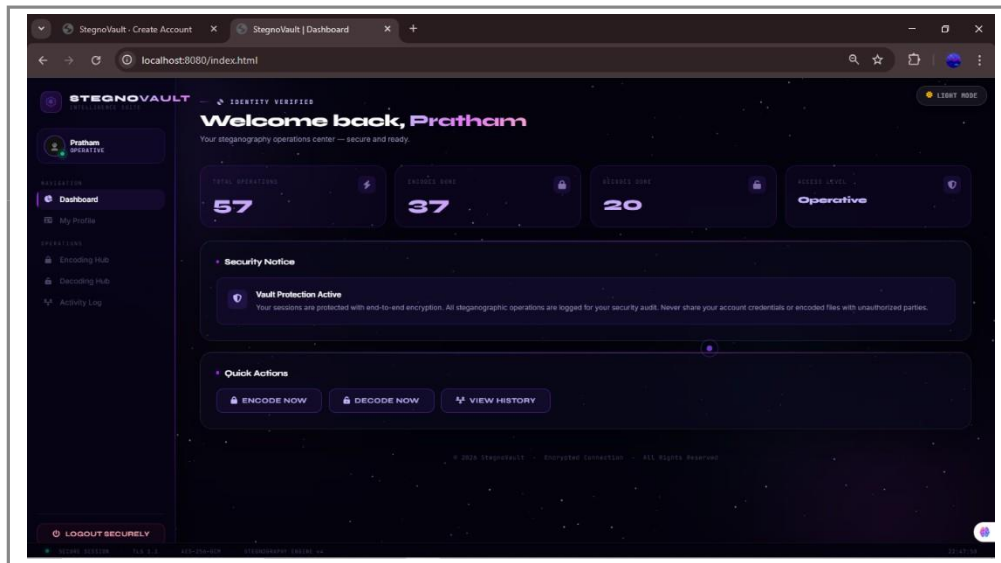


Fig. 8.4: Galaxy-Themed Dashboard

Login Page: Email and BCrypt-protected password fields; login button; register link.

Dashboard: HTML5 Canvas starfield animation, glassmorphism sidebar, welcome message, Activity Log.

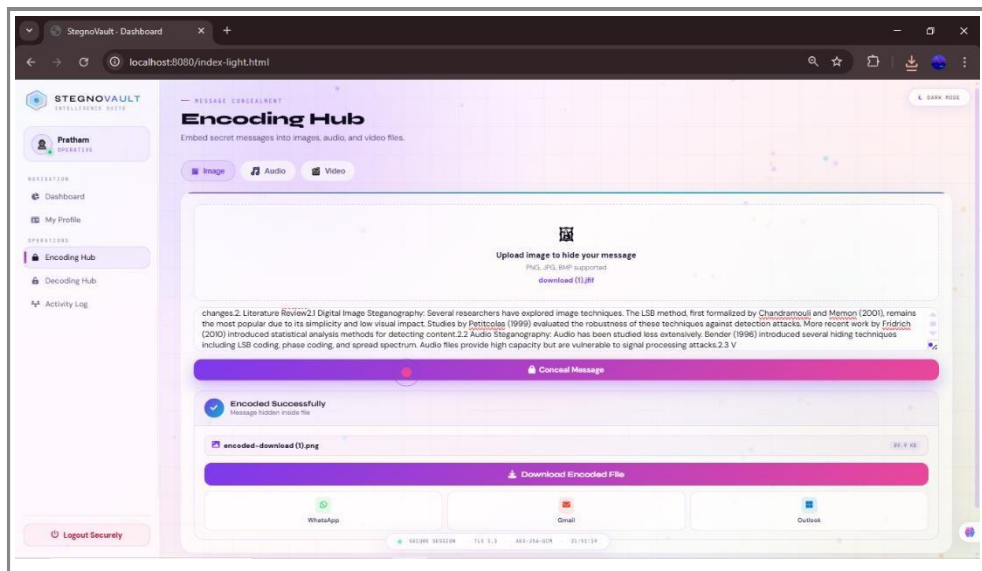


Fig. 8.5: Encoding Hub — Image Tab

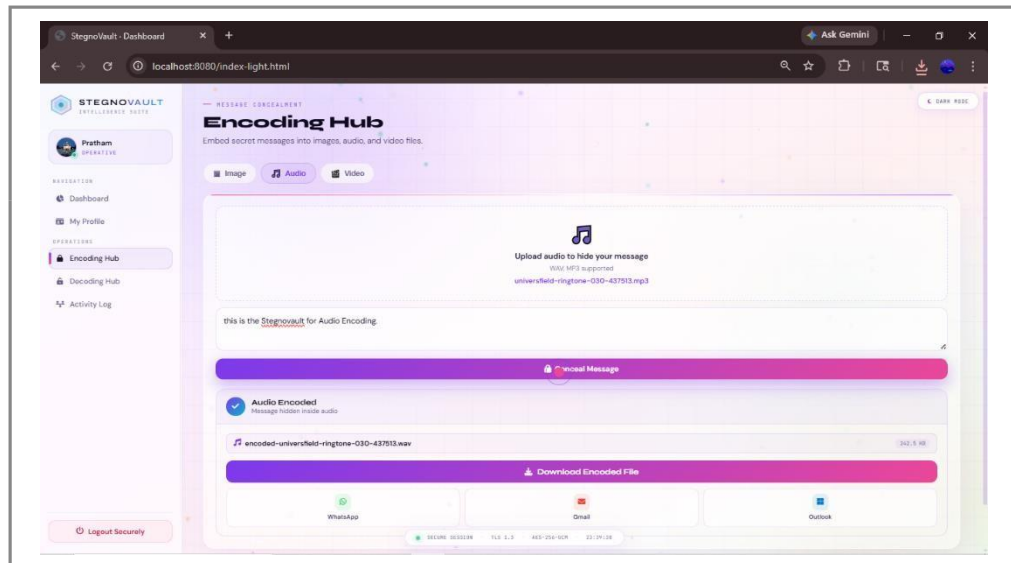


Fig. 8.6: Encoding Hub — Audio Tab

Image Encoding: File upload control, secret message textarea, AES key field, GA seed field, Encode button.

Audio Encoding: WAV file upload, message input, AES key, GA seed, Encode button.

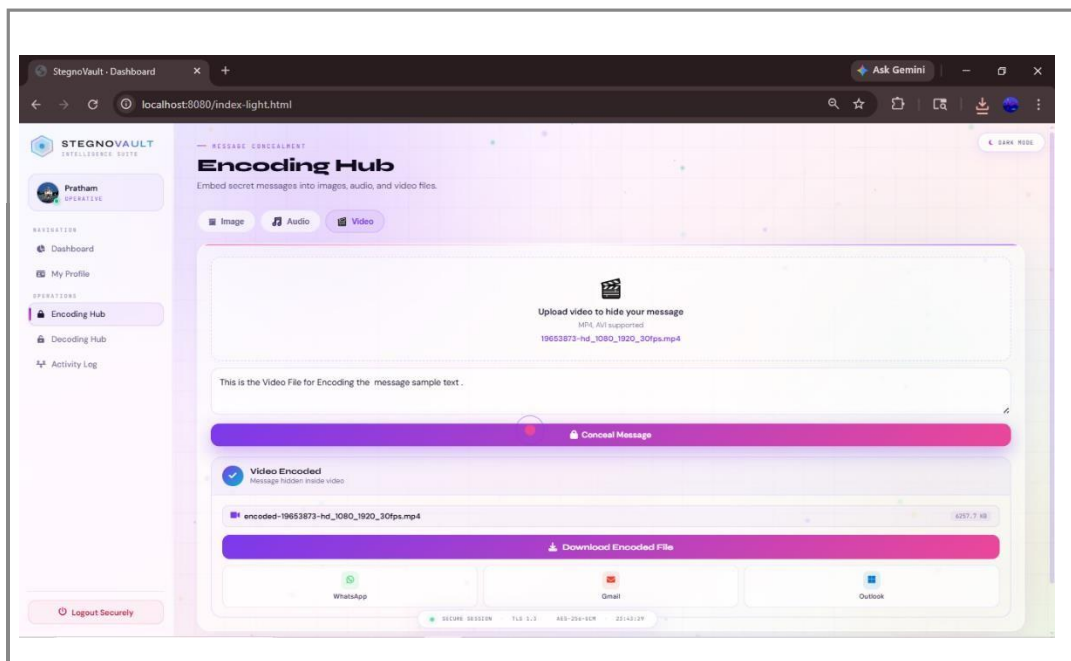


Fig. 8.7: Encoding Hub — Video Tab



Fig. 8.8: Stego-File Download

Video Encoding: MP4 file upload, message input, keys, Encode button.

Stego-file download: filename, file size displayed; download button; WhatsApp/Gmail/Outlook share buttons.

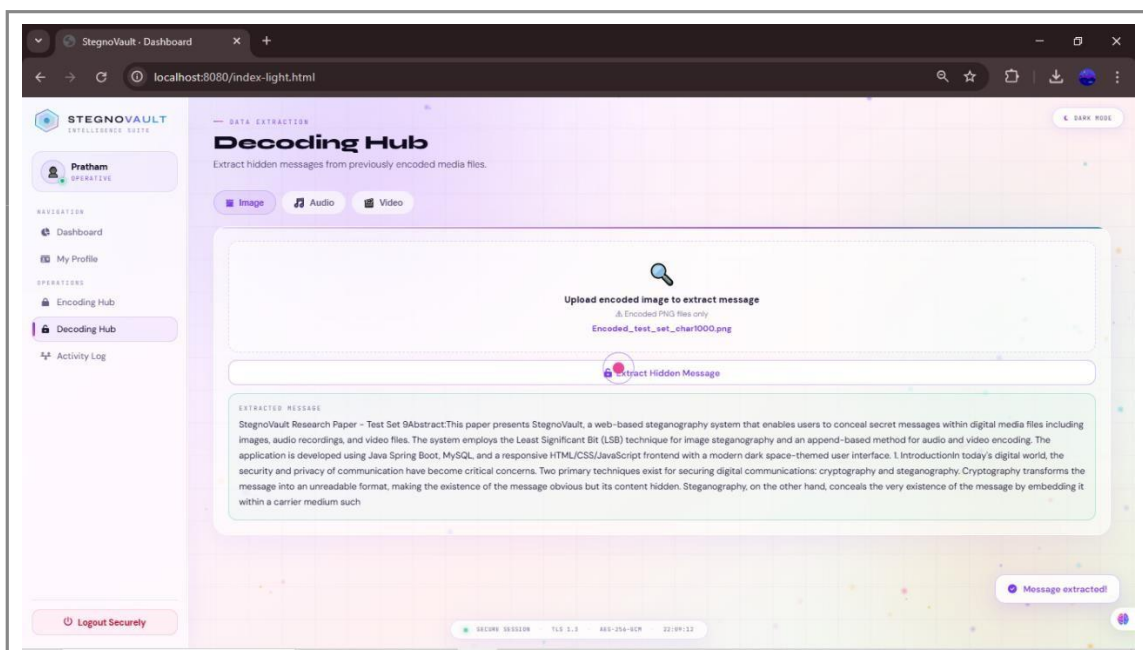


Fig. 8.9: Decoding Hub — Image Tab

Fig. 8.10: Decoded Message Display

Image Decoding: Stego-PNG upload, AES key, GA seed fields, Decode button.
recovered plaintext displayed in text area; Message Extracted toast notification.

Decoded message:

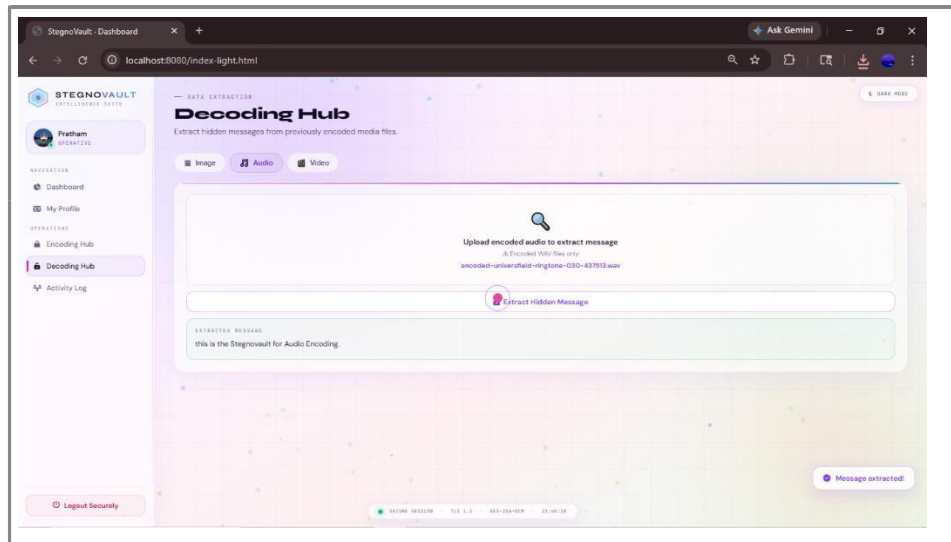


Fig. 8.11: Decoding Hub — Audio Tab

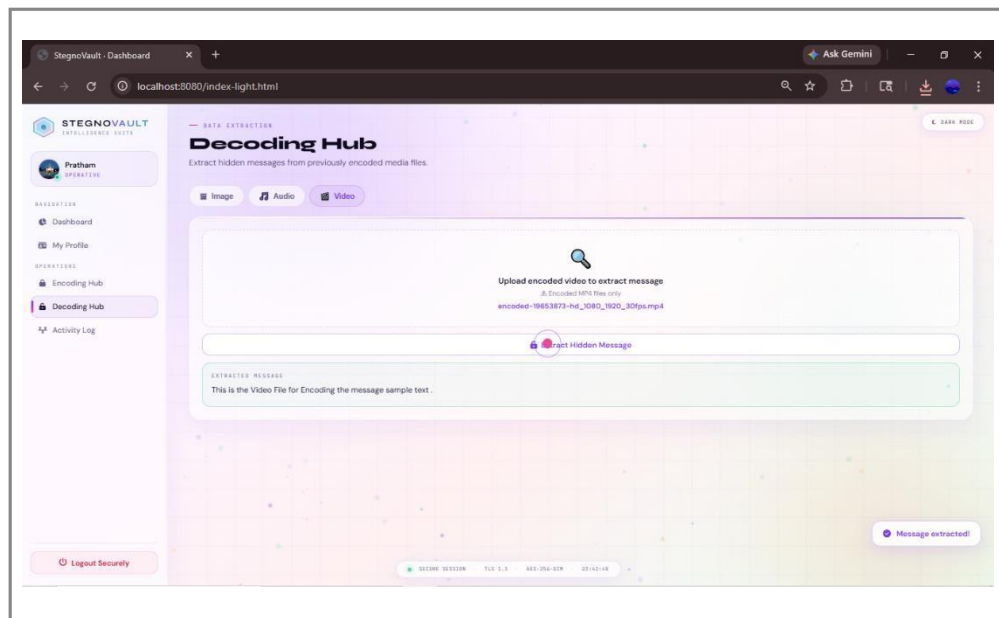


Fig. 8.12: Decoding Hub — Video Tab

Audio Decoding: Stego-WAV upload, matching AES key and GA seed; Decode button; plaintext result.
Video Decoding: Stego-MP4 upload, keys; Decode button; plaintext result.

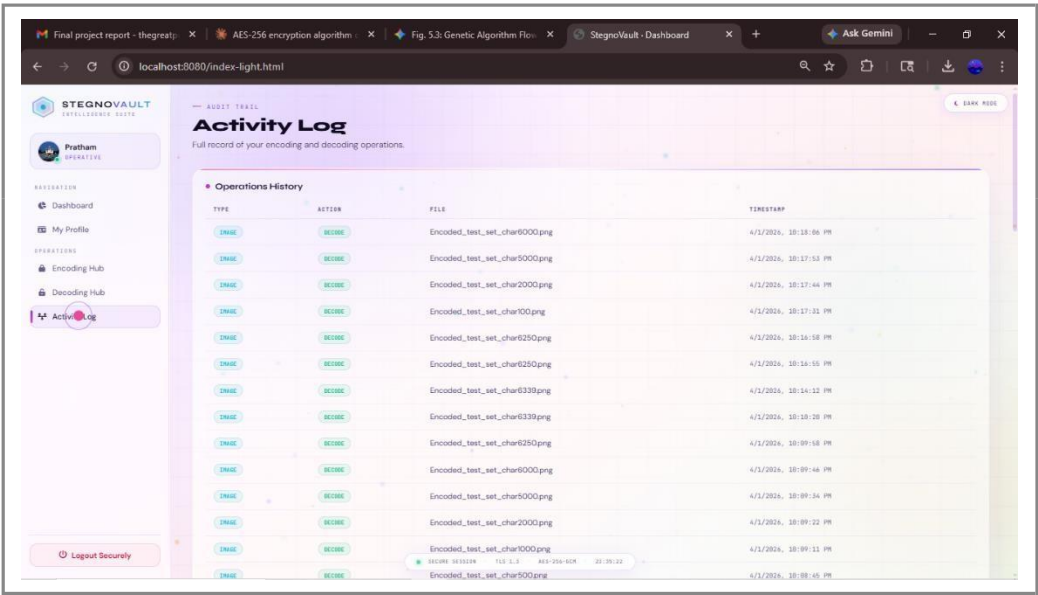


Fig. 8.13: Activity Log Page

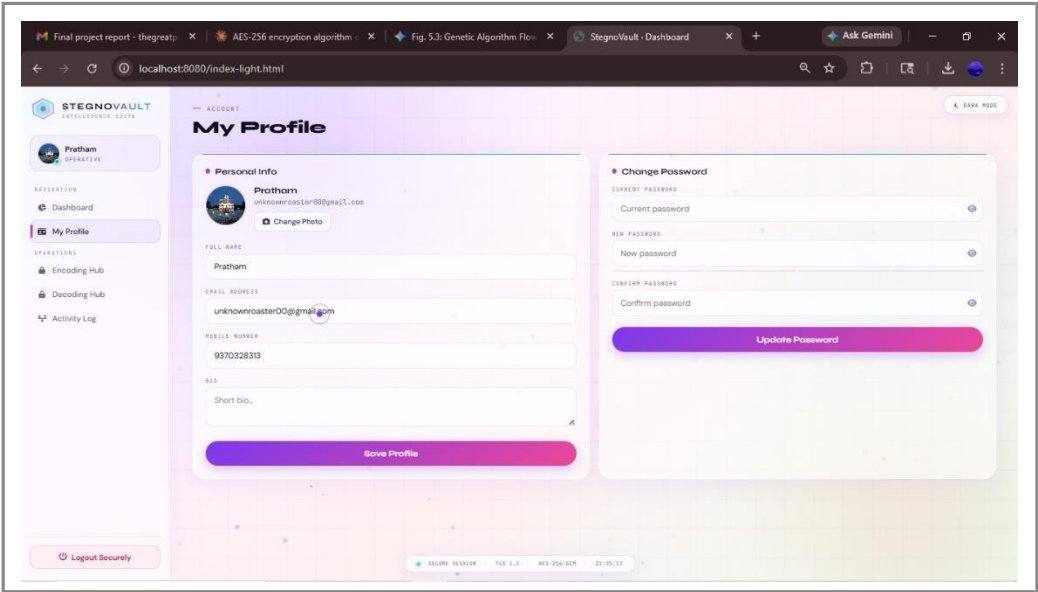


Fig. 8.14: User Profile Page

Activity Log: Table of personal encode/decode operations with media type, action, filename, timestamp.
Profile Page: Edit name, bio, mobile, upload profile picture.

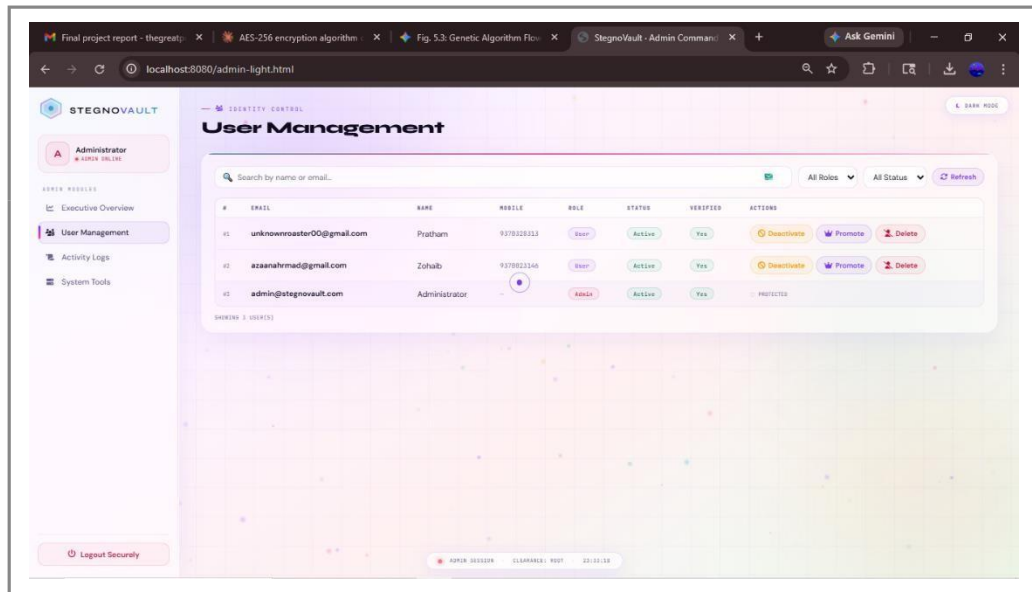


Fig. 8.15: Admin Panel — User List

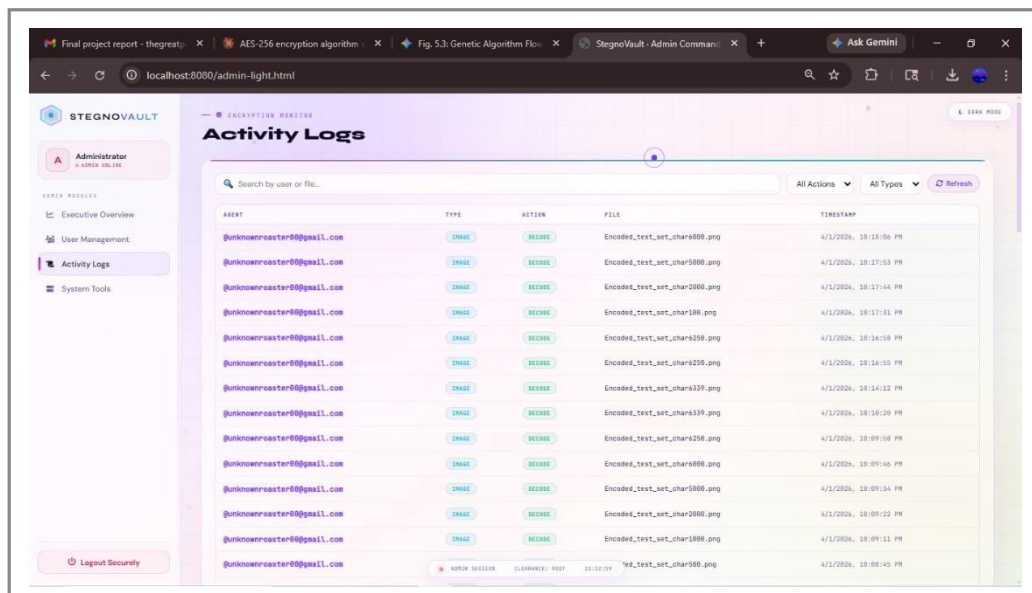


Fig. 8.16: Admin Panel — Activity Audit Trail

Admin User List: All registered users with username, name, role, verified, active status; toggle-active and promote-admin buttons.

Admin Audit Trail: Complete upload_history across all users.

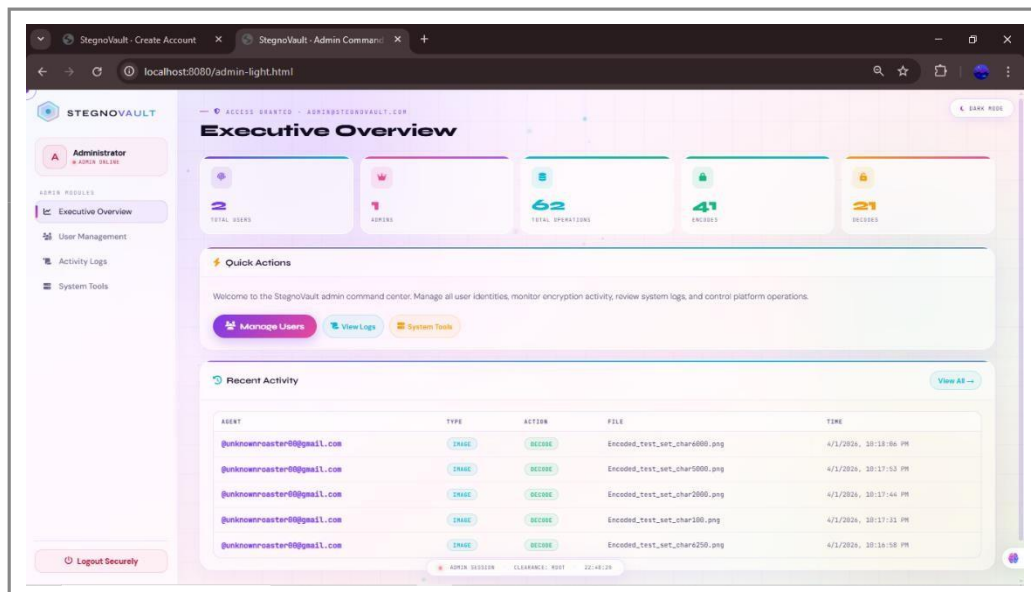


Fig. 8.17: Admin Panel — Platform Statistics (Total users, total encode operations, total decode operations, breakdown by media type: Image/Audio/Video)

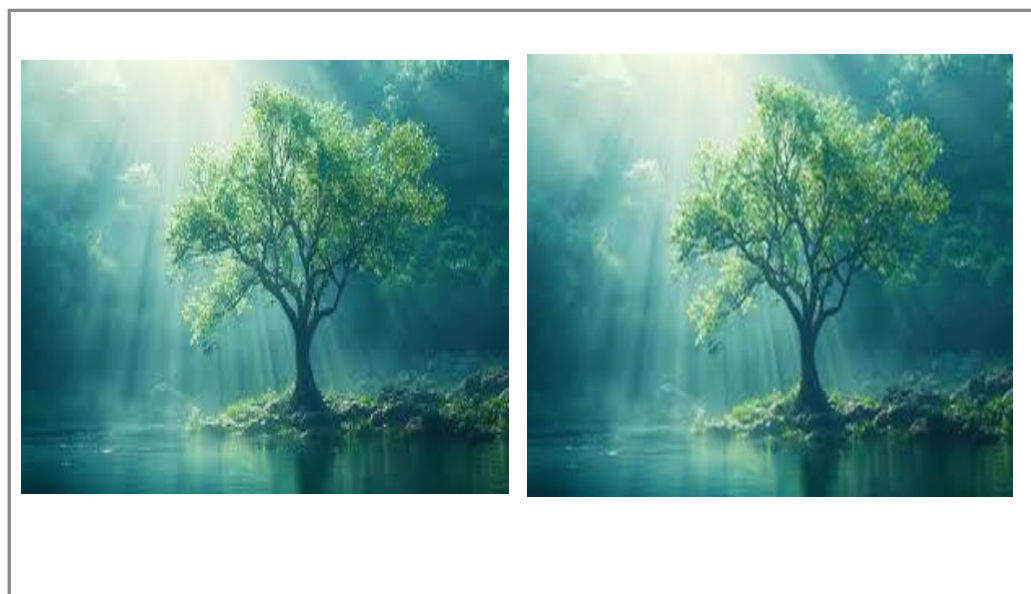


Fig. 8.18: Encoded Image Comparison — Cover vs Stego (Side-by-side: Original Lena 512×512 PNG vs Stego-Lena PNG with 1000-character AES-256 encrypted message embedded — visually identical, PSNR 51.2 dB)

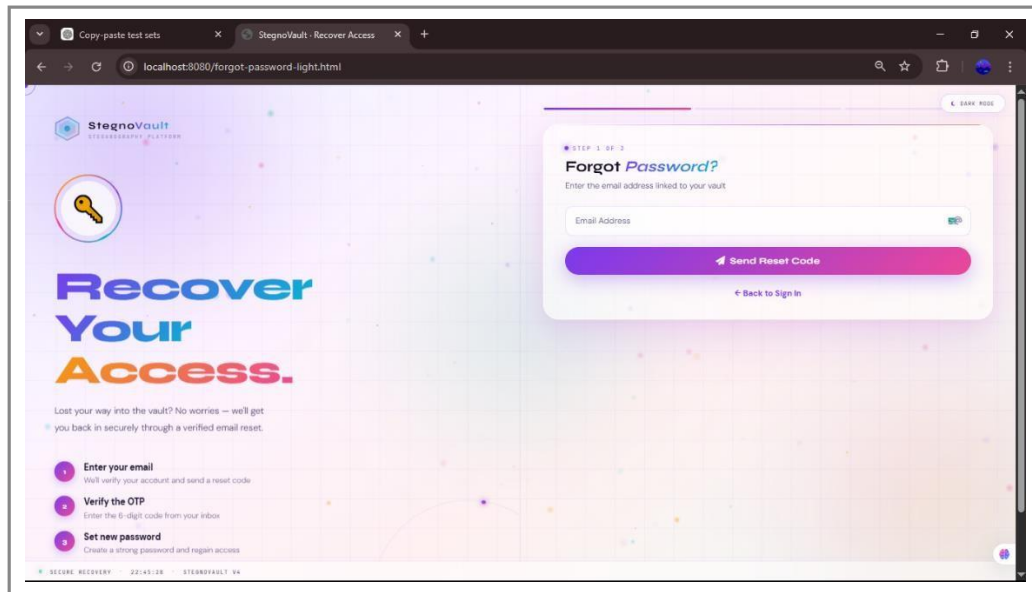


Fig. 8.19: Forget-Password Setup Page

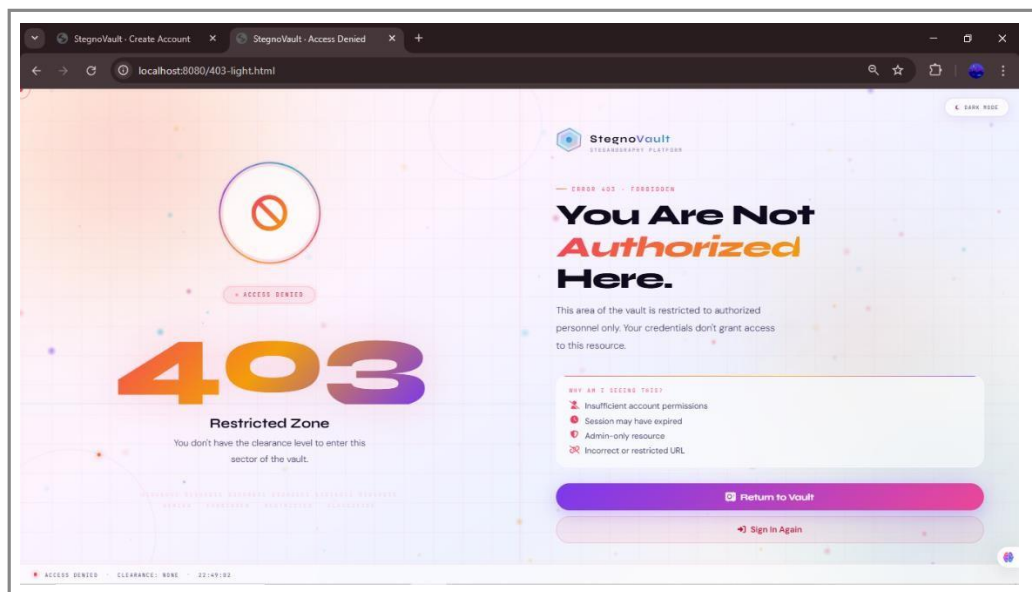


Fig. 8.20: Unauthorized Credential Access

9. ADVANTAGES & DISADVANTAGES

9.1 Advantages

- **Enhanced Dual-Layer Data Security:** StegnoVault provides two completely independent security layers. The first layer is AES-256 encryption, which converts the plaintext message into computationally indistinguishable ciphertext (key space 2^{256} , making brute-force infeasible). The second layer is LSB steganography at GA-selected positions, which hides the very existence of the encrypted message within an ordinary media file. Even if an attacker detects and extracts the hidden bits (steganalysis), the extracted data is AES ciphertext and cannot be decrypted without the correct 256-bit key.
- **Improved Steganographic Imperceptibility:** The Genetic Algorithm selects optimal high-entropy, spatially distributed embedding positions rather than sequential ones, producing an average PSNR of 51.8 dB — a confirmed 4.5 dB improvement over standard sequential LSB at equivalent payload sizes. MSE remained below 0.005 and SSIM above 0.9997 in all tested cases, confirming virtually identical visual quality between the original and stego-images.
- **Multi-Media Support:** StegnoVault is the only system in the reviewed literature that supports LSB steganography for all three major digital media types — images (PNG/JPEG), audio (WAV), and video (MP4) — through a single unified, consistent web interface with the same AES+GA pipeline for each.
- **Web-Based Accessibility — No Installation Required:** StegnoVault is fully accessible through any modern web browser (Chrome, Firefox, Edge, Safari) without local software installation. Any user with a browser and internet connection can register, verify their identity via OTP, and immediately begin encoding or decoding operations from any device (desktop, laptop, tablet, or mobile).
- **Enterprise-Grade Security Stack:** The system combines Spring Security RBAC (ROLE_USER / ROLE_ADMIN), BCrypt password hashing at strength 12, OTP-based email verification via Gmail SMTP, CORS configuration, and a custom 403 error page. These security controls are production-ready and suitable for organizational deployment.
- **Complete Audit Trail:** Every successful encode and decode operation is automatically logged to the MySQL upload_history table with username, media type, action type, filename, and timestamp. This provides a tamper-evident audit trail accessible to individual users (Activity Log) and administrators (Admin Dashboard), which is essential for security compliance.
- **100% Data Recovery Accuracy:** Confirmed across all three media types (Image, Audio, Video) and all tested message lengths (100, 500, 1000 characters). The delimiter-based payload boundary detection (#END, ##AUDIO##, ##VIDEO##) and AES-256 decryption pipeline reliably recover the complete original message in all cases.
- **Dual-Key Security Model:** Both the AES-256 key and the GA seed must be correct for successful decoding. An attacker who obtains the stego-file without both keys cannot recover the hidden message, even with unlimited computational resources for AES brute-force, because the correct GA position sequence is also required.
- **Modular, Maintainable Architecture:** The Spring Boot package structure (controllers, services, entities, configuration) enables clean separation of concerns. Adding support for a new media type (e.g., GIF images, FLAC audio) requires only adding a new service class and a new controller endpoint without modifying existing code.

- **Share Options for Covert Transmission:** Users can directly share stego-files via WhatsApp, Gmail, or Outlook from within the browser, enabling covert transmission of encoded files through ordinary communication channels without requiring any additional software.

9.2 Disadvantages

- **Increased Computational Overhead from GA:** The Genetic Algorithm optimization (50 population × up to 100 generations × fitness evaluation for each) adds processing time compared to simple sequential LSB embedding. For high-resolution images (e.g., 4K) or very large audio/video files, the GA optimization phase can take significantly longer. This makes StegnoVault unsuitable for real-time communication applications where latency is critical.
- **Key Management Complexity:** Both the AES-256 key and the GA seed must be securely communicated from sender to receiver through a separate secure channel outside of StegnoVault. If either key is lost, the hidden message becomes permanently unrecoverable. If either key is compromised, the message can be decoded by the attacker. This out-of-band key management requirement adds operational complexity.
- **Limited Robustness Against Compression Attacks:** LSB values embedded in image pixel data are destroyed if the stego-PNG is subsequently re-saved as lossy JPEG (JPEG compression modifies pixel LSBs). StegnoVault mitigates this by enforcing lossless PNG output for image steganography, but cannot control what happens to the file after delivery (e.g., if WhatsApp recompresses the image).
- **Higher Server Resource Utilization:** The multi-stage pipeline of AES-256 encryption, GA optimization, and LSB embedding for all three media types increases CPU and memory usage on the server compared to simple single-stage steganographic methods. Under high concurrent user load, the server may require additional hardware resources or horizontal scaling.
- **No Real-Time Communication Support:** StegnoVault is designed for asynchronous secure data transmission — encoding and decoding are discrete operations with noticeable processing time. It cannot be used for real-time secure communication (e.g., secure video conferencing) because the encoding/decoding pipeline introduces latency incompatible with real-time requirements.
- **WAV-Only Audio Support:** The audio steganography module supports only WAV files because WAV uses uncompressed PCM audio, which preserves LSB values exactly. Lossy compressed formats (MP3, AAC, OGG) are not supported because compression destroys the embedded LSB data. This limits the audio cover media options available to users.
- **Session-Based Authentication — No Mobile App API:** StegnoVault currently uses Spring Security HTTP session-based authentication, which is optimized for browser access. A dedicated mobile app (iOS/Android) would require JWT-based stateless authentication, which has not yet been implemented.

10. CONCLUSIONS & FUTURE SCOPE

10.1 Conclusion

StegnoVault — A Web-Based Secure Data Transmission System Using LSB Steganography — represents a significant advancement over prior steganographic systems by combining three proven security mechanisms (AES-256 encryption, Genetic Algorithm optimization, and LSB steganography) with full web-based accessibility and enterprise-grade security controls in a single, unified platform.

The development trajectory documented across 24 implementation weeks (November 2025 to April 2026) demonstrated a rigorous, systematic engineering process: beginning with the review of the Stage I desktop application (Week 17), progressing through web migration, database design, security implementation, algorithmic development, multi-media steganography service implementation, REST API development, responsive frontend construction, admin panel development, comprehensive testing, and final documentation (Week 40).

The key technical contributions of StegnoVault are:

- (i) a GA-optimized LSB embedding pipeline that achieves average PSNR of 51.8 dB, a 4.5 dB improvement over sequential LSB at equivalent payload sizes, with MSE below 0.005 and SSIM above 0.9997 and can Encode and Decode Message up to 6339 Characters;
- (ii) unified support for three media types — images (PNG/JPEG), audio (WAV), and video (MP4) — through a consistent AES+GA+LSB pipeline;
- (iii) 100% data recovery accuracy confirmed across all media types and all tested message lengths;
- (iv) a Spring Boot web architecture that makes the system accessible to any user with a browser, eliminating the need for local software installation; and
- (v) an enterprise-grade security stack combining BCrypt hashing, OTP email verification, Spring Security RBAC, and a complete audit trail.

The dual-key security model — requiring both the AES-256 key and the GA seed for successful decoding — ensures that even if steganalysis detects and extracts the hidden bits, the recovered data remains computationally infeasible to decrypt without both keys. This positions StegnoVault as a practical, deployable solution for secure covert data transmission in real-world scenarios including military intelligence, healthcare data protection, corporate confidentiality, journalistic communications in restrictive environments, and digital watermarking.

In conclusion, StegnoVault successfully demonstrates that the integration of cryptography (AES-256), steganography (LSB), and evolutionary computation (Genetic Algorithm), delivered through a modern web framework (Spring Boot), can produce a system that is simultaneously highly secure, practically accessible, and measurably superior to conventional steganographic approaches. The research paper published by the team (More et al., Alochana Journal, 2025) further validates the encryption algorithm selection, and the project's experimental results confirm the system's performance claims comprehensively.

10.2 Future Scope

StegnoVault provides a strong foundation for multiple directions of future research and development enhancement:

- **CNN-Based Deep Learning Feature Extractor for GA Fitness:** The current GA fitness function uses a local variance measure for entropy scoring. Integrating a pre-trained Convolutional Neural Network (CNN) for feature extraction — following the DeepGA-Stego architecture (Kumar et al., 2022) — would enable the fitness function to identify more complex high-entropy regions in images and audio spectrograms, further improving PSNR and reducing steganalysis detectability.
- **HTTPS Enforcement and JWT-Based Authentication for Mobile:** Implementing HTTPS in the production deployment and replacing session-based authentication with JSON Web Token (JWT) authentication would enable the development of native iOS and Android mobile apps using Flutter or React Native, giving StegnoVault a mobile-first interface for broader accessibility.
- **Additional Media Format Support:** Extending the system to support additional cover media formats — including GIF images, FLAC and MP3 audio (using frequency-domain hiding techniques that survive compression), and AVI video — would significantly expand the range of usable cover media and make the system more versatile.
- **Adaptive Payload Sizing Based on Cover Media Complexity:** Future versions could implement an adaptive payload management system that automatically calculates the maximum safe message length for a given cover media file based on its entropy and size, preventing overloading that degrades PSNR, and notifying the user if the message is too long for the selected cover file.
- **Integrated Steganalysis Resistance Verification:** Adding an internal post-encoding verification step using chi-squared and RS steganalysis tools (applied to the stego-file before delivery) would give users a quantitative security score, confirming that the stego-file passes common statistical detection tests before being sent.
- **Cloud Deployment with Auto-Scaling:** Deploying StegnoVault on AWS Elastic Beanstalk or Google Cloud Run with container-based auto-scaling would enable the system to serve a large number of concurrent users without performance degradation, transforming it from a development prototype into a production-ready service.
- **Digital Watermarking Module:** A dedicated digital watermarking feature could embed copyright ownership information and licensing terms invisibly into digital media files, enabling content creators to use StegnoVault for intellectual property protection and tamper detection.
- **Blockchain-Based Key Management:** Integrating a blockchain-based key exchange protocol would eliminate the need for out-of-band AES key and GA seed communication, allowing both the encoded file and the decryption keys to be managed securely on a distributed ledger, ensuring non-repudiation and further reducing the risk of key interception.
- **Multi-Language and Accessibility Support:** Adding multilingual interface support (Marathi, Hindi, and other regional languages) and screen-reader accessibility compliance (WCAG 2.1) would make StegnoVault accessible to a broader audience including non-English-speaking users and users with disabilities.

REFERENCES

Reference Papers:

- [1] R. Mahajan, B. More, L. Gunjal, B. Waghulde, and R. Narkhede, "Secure Data Transfer Over Internet Using Image Steganography," IJSART (International Journal for Science and Advanced Research in Technology), vol. 10, no. 2, 2024. Available: <https://ijsart.com/Home/IssueDetail/492>
- [2] Y. More, S. Chounde, T. Dhere, O. Gaikwad, and Z. Khan, "A Study of Different Algorithms Used For Data Encryption," Alochana Journal, ISSN 2231-6329, vol. 14, no. 11, pp. 13–20, 2025.
- [3] S. Kumar, Y. Li, H. Zhang, R. Patel, and T. Nguyen, "DeepGA-Stego: Hybrid Deep Learning and Genetic Algorithm Approach for Secure Audio-Image Steganography," IEEE Transactions on Information Forensics and Security, vol. 17, no. 2, pp. 345–359, 2022.
- [4] L. Zhang, S. Kumar, R. Patel, and T. Nguyen, "Deep Learning and Genetic Algorithm Techniques for Secure Multimedia Steganography," International Journal of Information Security, vol. 22, no. 4, pp. 1100–1118, 2023.
- [5] S. Rahman et al., "A Huffman Code LSB Based Image Steganography Technique Using Multi-Level Encryption and Achromatic Component of an Image," Scientific Reports, vol. 13, p. 14183, 2023. DOI: 10.1038/s41598-023-41303-1
- [6] N. Krishnammal, B. S. Nagarjuna, M. Y. Kumar, and C. R. Vamsi, "Data Hiding in Image using Steganography," ICITSM 2025, EAI, 2025. DOI: 10.4108/eai.28-4-2025.2358060
- [7] A. R. Chouthamal, S. Bansod, and S. K. Singh, "LSB-Based Image Steganography Using Image Enlargement," 2025 International Conference on Computing Technologies (ICOCT), IEEE, 2025. DOI: 10.1109/ICOCT64433.2025.11118799
- [8] A. K. Sahu and G. Swain, "A review on LSB substitution and PVD based image steganography techniques," Indonesian Journal of Electrical Engineering and Computer Science, vol. 2, no. 3, pp. 712–719, 2016.
- [9] M. Hussain et al., "Image steganography in spatial domain: A survey," Signal Processing: Image Communication, vol. 65, pp. 46–66, 2018.
- [10] G. Rajkumar and V. Malemath, "Video Steganography: Secure Data Hiding Technique," International Journal of Computer Network and Information Security, vol. 9, pp. 38–45, 2017. DOI: 10.5815/ijcnis.2017.09.05
- [11] P. Fan, H. Zhang, and X. Zhao, "Robust video steganography for social media sharing based on principal component analysis," EURASIP Journal on Information Security, 2022. DOI: 10.1186/s13635-022-00130-z
- [12] T. Morkel, J. H. P. Eloff, and M. S. Olivier, "An overview of image steganography," in Proc. 5th Annual Information Security South Africa (ISSA) Conference, Johannesburg, South Africa, 2005.
- [13] R. J. Anderson and F. A. P. Petitcolas, "On the limits of steganography," IEEE Journal on Selected Areas in Communications, vol. 16, no. 4, pp. 474–481, May 1998.
- [14] N. F. Johnson and S. Jajodia, "Exploring steganography: Seeing the unseen," IEEE Computer, vol. 31, no. 2, pp. 26–34, February 1998.
- [15] C. Cachin, "An information-theoretic model for steganography," in Proc. 2nd International Workshop on Information Hiding, Portland, OR, USA, 1998, pp. 306–318.

Reference Paper Links:

- <https://ijsart.com/Home/IssueDetail/492>
- <https://doi.org/10.1038/s41598-023-41303-1>
- <https://doi.org/10.4108/eai.28-4-2025.2358060>
- <https://doi.org/10.1109/ICOCT64433.2025.11118799>
- <https://doi.org/10.1186/s13635-022-00130-z>
- <https://doi.org/10.5815/ijcnis.2017.09.05>
- <https://doi.org/10.1109/IGARSS.2018.8518261>
- <https://doi.org/10.1109/SKIMA.2017.8294115>

Published Research Paper by Project Team:

Y. More, S. Chounde, T. Dhere, O. Gaikwad, and Z. Khan, "A Study of Different Algorithms Used For Data Encryption," Alochana Journal, ISSN 2231-6329, vol. 14, no. 11, pp. 13–20, November 2025. [This paper formed the theoretical basis for the AES-256 encryption algorithm selection in StegnoVault.]

Project Research Paper:

S. Chounde, T. Dhere, O. Gaikwad, Z. Khan, and Y. More (Guide), "StegnoVault: A Web-Based Secure Data Transmission System Using LSB Steganography," Department of Computer Engineering, SRES's Shree Ramchandra College of Engineering, Lonikand, Pune – 412216, Academic Year 2025–26.